Documentazione ingegneria del software

Progetto: Webapp per la gestione della glicemia

Fazion Pietro (VR501347) , Dal Degan Pietro (VR500305)

Indice

1 Azioni degli utenti **3**
1.1 Casi d'uso del Paziente . . . 4
1.1.1 inserimento della rilevazione della glicemia . . . 5
1.1.2 Segnalazione di patologie, malattie e/o sintomi . . . 6
1.1.3 Registrazione giornaliera di medicinali assunti . . . 7
1.1.4 Contattare il medico . . . 8
1.2 Casi d'uso del Medico . . . 9
1.2.1 Visualizza alert per il medico . . . 10
1.2.2 Gestione Terapie . . . 11
1.2.3 Gestione Pazienti . . . 15
1.3 Casi d'uso dell'Admin . . . 19
1.3.1 Visualizza log . . . 20
1.3.2 Gestisci Pazienti . . . 21
1.3.3 Gestisci Medici . . . 26

2 Diagrammi di attività **30**
2.1 Diagramma di autenticazione . . . 30
2.2 Diagramma del paziente . . . 31
2.2.1 Diagramma per la segnalazione di patologie, malattie e/o sintomi . . . 32
2.2.2 Diagramma di come un paziente comunica con il medico . . . 33
2.2.3 Diagramma per l'inserimento della glicemia . . . 34
2.2.4 Diagramma per la registrazione giornaliera dei medicinali . . . 35
2.3 Diagramma del medico . . . 36
2.3.1 Diagramma di gestione delle terapie . . . 37
2.3.2 Diagramma di visualizzazione degli alert del medico . . . 38
2.3.3 Diagramma di gestione dei pazienti . . . 39
2.4 Diagramma dell'admin . . . 40
2.4.1 Diagramma gestione dei pazienti . . . 41
2.4.2 Diagramma della gestione dei medici . . . 42
2.4.3 Diagramma visualizzazione dei log . . . 43

3 Diagrammi UML **44**
3.1 Diagramma delle classi - Package Entity . . . 44
3.1.1 Entità principali . . . 45
3.2 Diagrammi di sequenza UML . . . 46

3.2.1 Autenticazione (Sign-In) . . . 46
3.2.2 Registrazione utente . . . 48
3.2.3 Creazione terapia . . . 49
3.2.4 Inserimento rilevazione con alert automatico . . . 50
3.2.5 Filtro JWT per autenticazione richieste . . . 51
3.2.6 Logout . . . 53
3.2.7 Controllo automatico terapie (Scheduled Task) . . . 54

4 Sviluppo del Software **56**
4.1 Metodologia di sviluppo . . . 56
4.2 Architettura . . . 57
4.2.1 Configurazione del Database . . . 57
4.3 Pattern utilizzati . . . 58
4.3.1 Singleton . . . 58
4.3.2 Repository Pattern . . . 58
4.3.3 Observer Pattern . . . 58
4.3.4 Facade Pattern . . . 58
4.3.5 Builder Pattern . . . 59
4.4 Implementazione . . . 59
4.4.1 Struttura del Backend (Spring Boot) . . . 59
4.4.2 Repository Layer . . . 59
4.4.3 Service Layer . . . 60
4.4.4 Controller Layer . . . 60
4.4.5 Autenticazione e Sicurezza . . . 61
4.4.6 Comunicazione Real-time con WebSocket . . . 61
4.4.7 Data Transfer Objects (DTO) . . . 62
4.4.8 Gestione delle eccezioni . . . 62
4.4.9 Logging . . . 62
4.4.10 Struttura del Frontend (React + TypeScript) . . . 63

5 Test **65**
5.1 Test automatici . . . 65
5.2 Test manuali . . . 66
5.2.1 Test di un utente reale . . . 66

Chapter 1

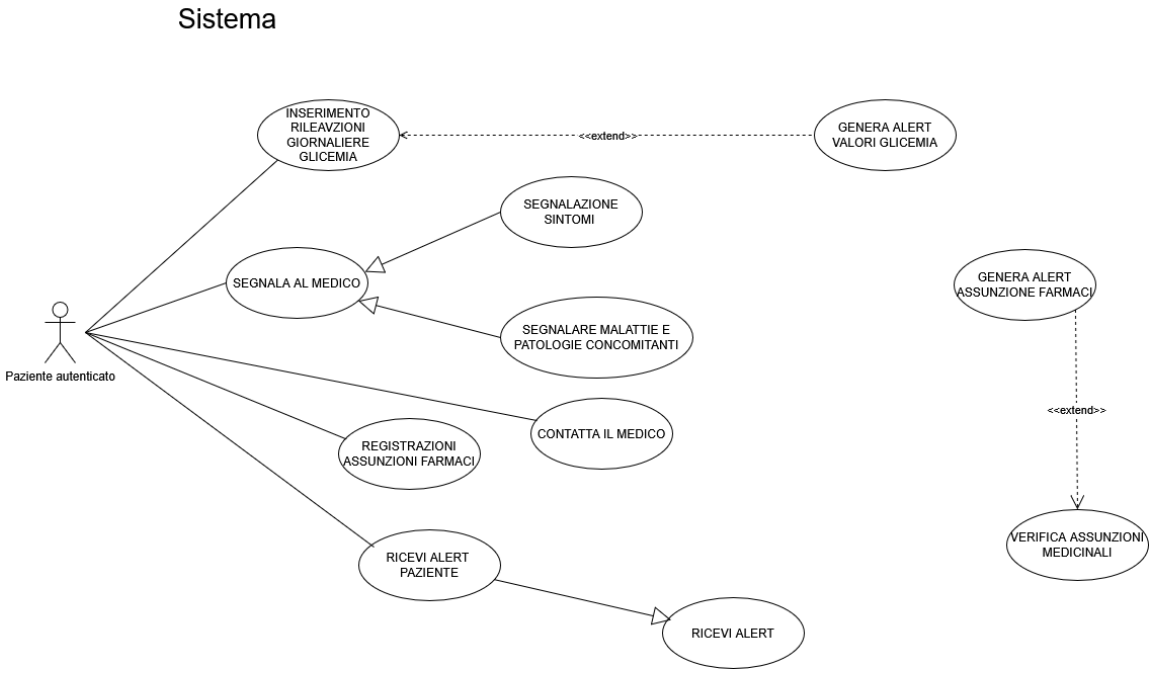
Azioni degli utenti

La Webapp che siamo andati a sviluppare è a servizio di tre tipologie di utenti:

- Paziente
- Medico
- Admin

Tutte le precedenti categorie di utenti hanno bisogno di autenticarsi per poter accedere alla propria area riservata. Andiamo a vedere ora i casi d'uso e i loro diagrammi di sequenza.
Per motivi di chiarezza nella visualizzazione delle immagini, divideremo i casi d'uso per utente, nonostante siano collegati tra loro.

1.1 Casi d'uso del Paziente



Dopo essersi autenticato il paziente verrà reindirizzato alla sua homepage, dove troverà:

- Un grafico con l'andamento della sua glicemia nel tempo
- Un pulsante per l'inserimento della glicemia
- Gli ultimi record di glicemia inseriti
- Un form per la comunicazione di sintomi e malattie
- Un area per contrassegnare i medicinali assunti
- Un form per contattare il medico

Ora elenchiamo di seguito i casi d'uso visti nell'immagine:

- Inserimento della rilevazione della glicemia
- Segnalazione di patologie, malattie e/o sintomi riscontrati al medico
- Registrazione giornaliera di medicinali assunti
- Contattare il medico

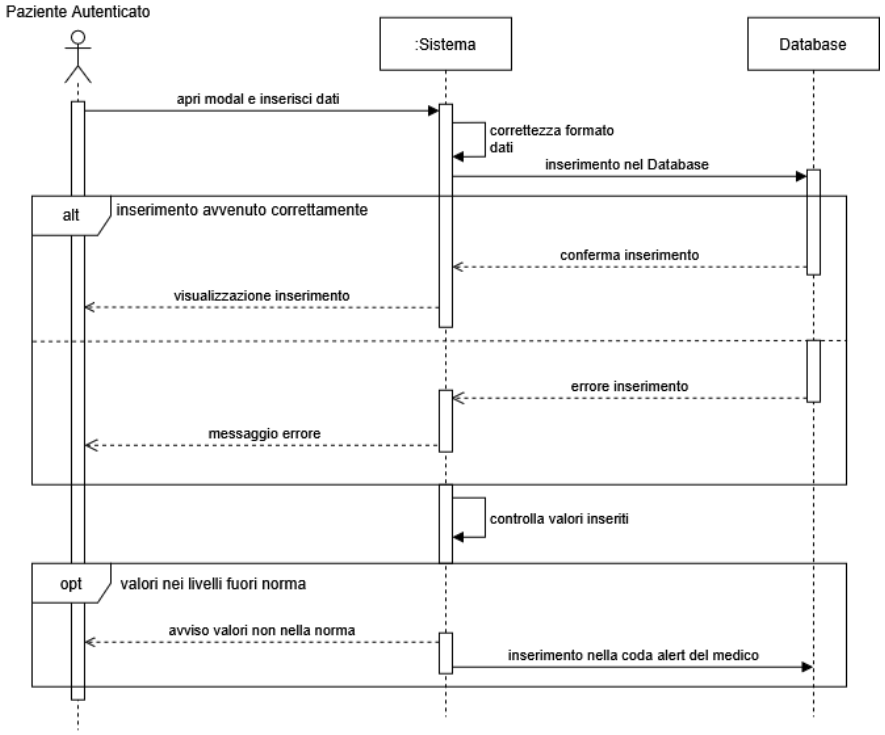
1.1.1 inserimento della rilevazione della glicemia

Il paziente deve essere in grado di poter inserire il livello glicemico, schiacciando il pulsante "inserisci glicemia" si visualizza un modal dove inserire il proprio livello glicemico. In particolare quando un paziente inserisce una rilevazione che non rientra nel range normale (80-130 mg/dL e massimo 180mg/dL dopo i pasti) il sistema registra la rilevazione e genera l'alert per informare il medico.

Attori: Paziente
Precondizioni: Il paziente deve essersi autenticato
Passi:

1. Il paziente clicca il pulsante "Nuova rilevazione"
2. Il paziente riempie il campo del modal glicemia che compare
3. Il paziente clicca il pulsante "invia"
4. Viene controllata la validità del dato inserito
5. Eventualmente vengono creati alert per Medico e Paziente in caso di valori fuori range

Postcondizioni: Il dato glicemico viene inserito nel database



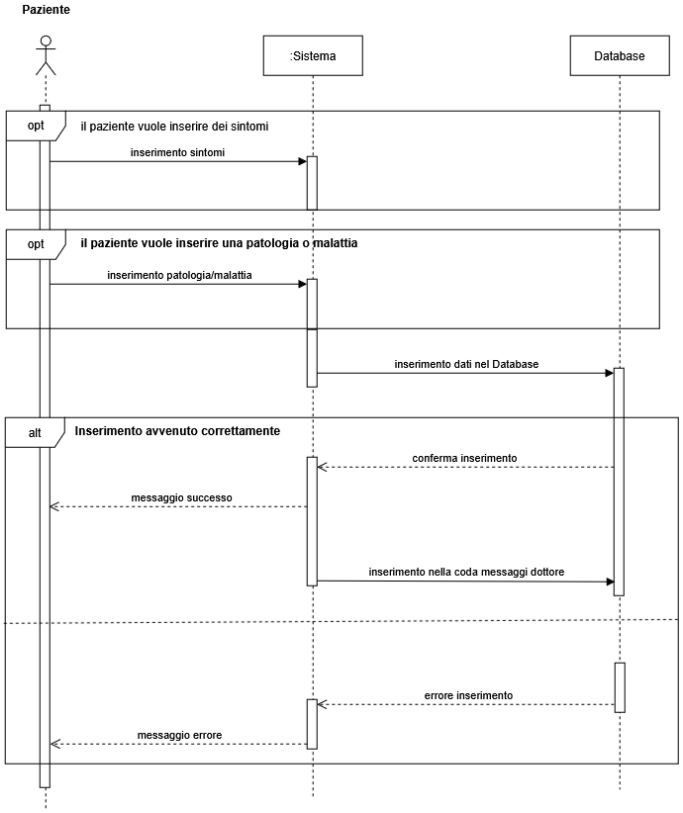
1.1.2 Segnalazione di patologie, malattie e/o sintomi

Il paziente può segnalare al proprio medico eventuali patologie, malattie e/o sintomi. Nel riquadro dedicato il paziente può selezionare i vari sintomi dalle checkbox oppure scriverli liberamente nel campo di testo assieme a eventuali malattie e patologie.

Attori: Paziente
Precondizioni: Il paziente deve essersi autenticato
Passi:

1. Il paziente seleziona le checkbox dei sintomi riscontrati
2. Il paziente compila il campo di testo inserendo patologie e malattie
3. Il paziente clicca il pulsante "invia segnalazione"
4. La segnalazione viene comunicata al medico

Postcondizioni: La segnalazione viene inserita nel database e viene segnalata al medico



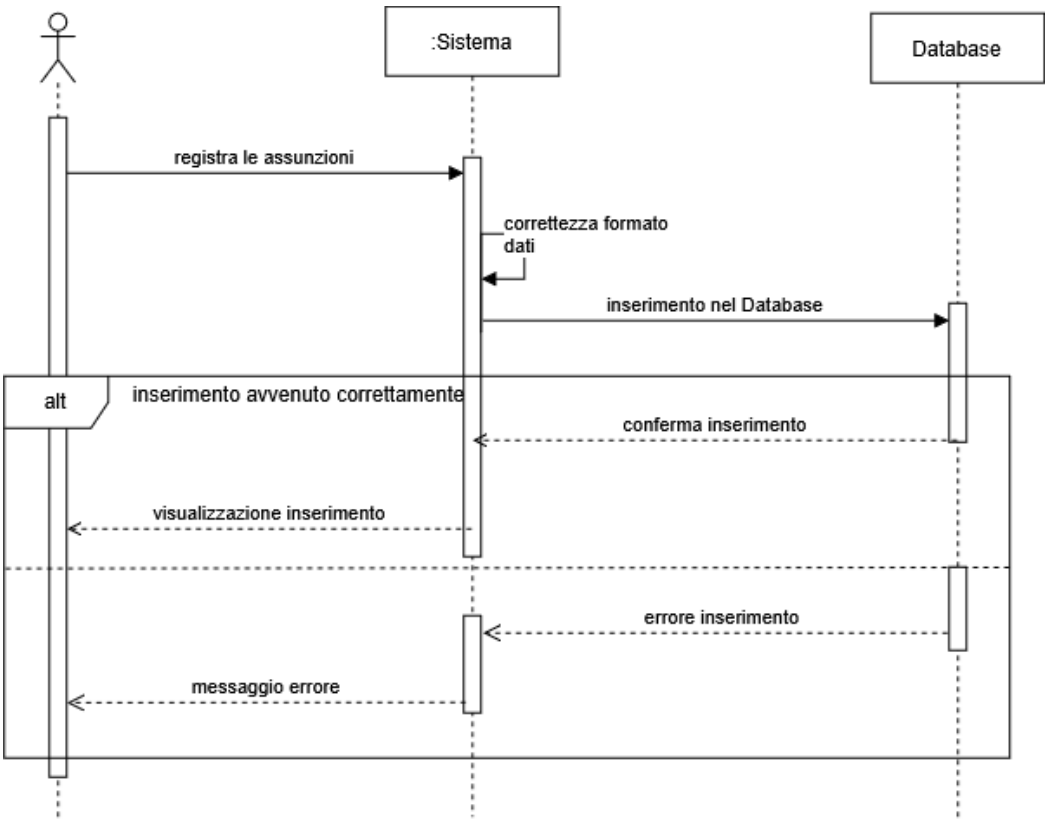
1.1.3 Registrazione giornaliera di medicinali assunti

Il paziente può registrare i medicinali assunti giornalmente. Nel riquadro apposito ci sarà la lista dei medicinali che il paziente dovrebbe assumere durante la giornata, dovrà semplicemente selezionare quelli assunti e schiacciare il pulsante "invia report al medico" affinchè possano essere registrati. Inoltre si potrà visualizzare il numero di volte che si è assunto un certo medicinale in un giorno.

Attori: Paziente
Precondizioni: Il paziente deve essersi autenticato
Passi:

1. Il paziente seleziona i medicinali assunti dalla lista
2. Il paziente clicca il pulsante "invia report al medico"
3. Vengono poi controllati i dati inseriti
4. Infine viene confermata l'avvenuta registrazione dei medicinali o meno

Postcondizioni: I medicinali assunti vengono registrati nel database



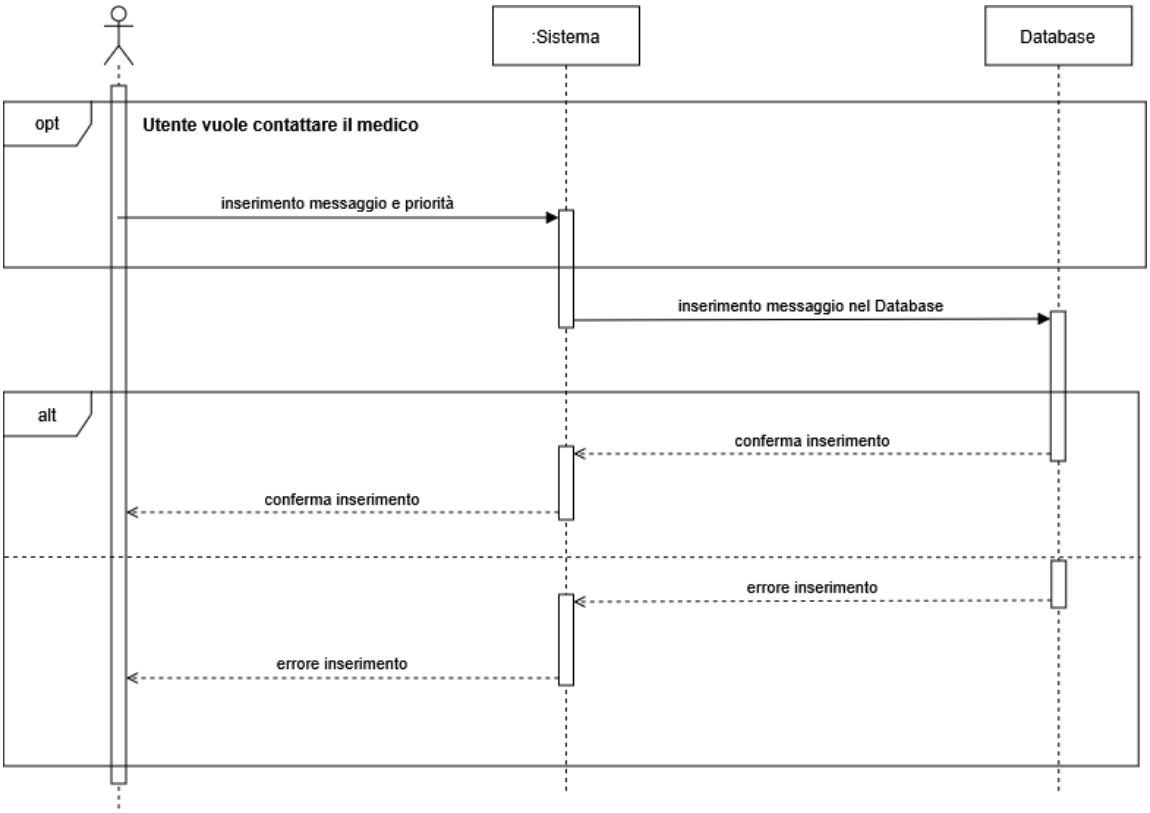
1.1.4 Contattare il medico

Il Paziente può contattare il proprio medico grazie ad un form dedicato. Nel riquadro apposito il paziente dovrà compilare il campo di testo e selezionare la priorità del messaggio tra bassa, media e alta; schiacciando poi il tasto "invia" per inviare il messaggio al medico.

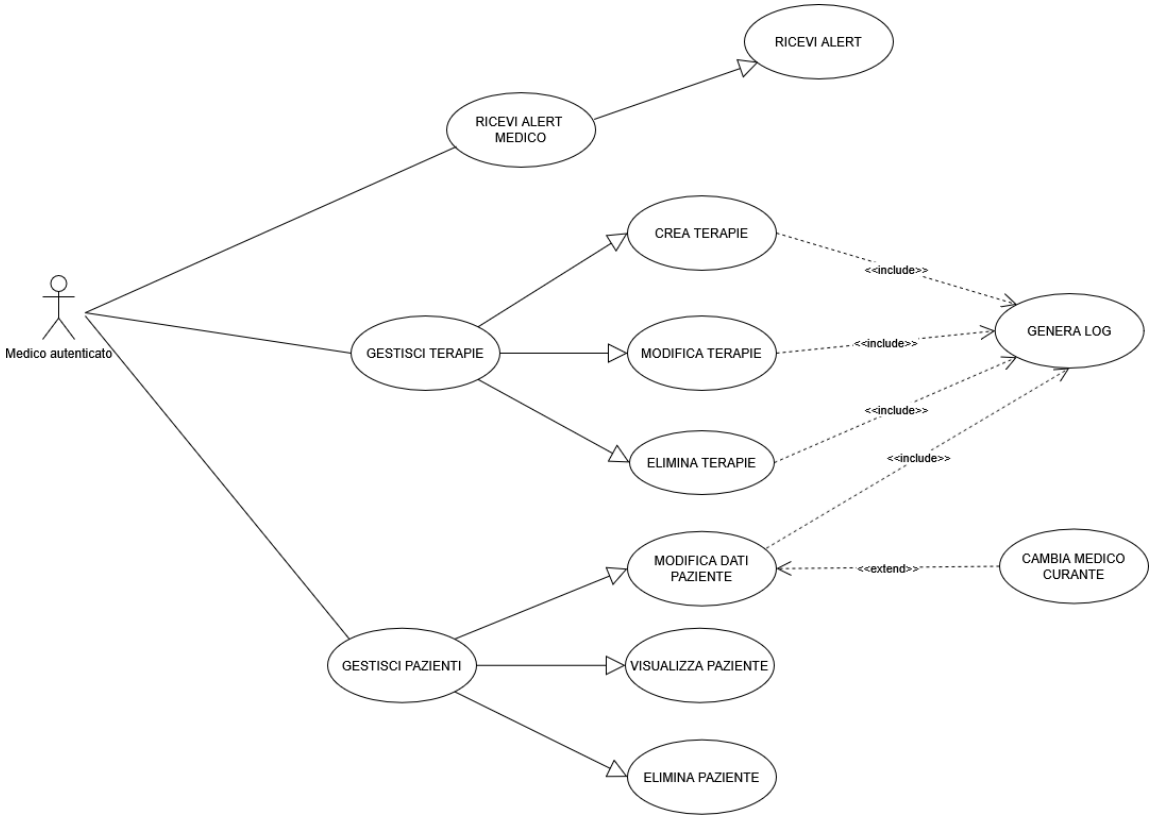
Attori: Paziente
Precondizioni: Il paziente deve essersi autenticato
Passi:

1. Il Paziente compila il campo di testo
2. Il paziente seleziona la priorità
3. Il paziente clicca il pulsante "invia"
4. Viene data conferma dell'avvenuto invio del messaggio o meno

Postcondizioni: Il messaggio viene inviato al medico



1.2 Casi d'uso del Medico



Dopo essersi autenticato il medico verrà reindirizzato alla sua homepage, dove troverà:

- Un riquadro con la lista di messaggi e alert ricevuti
- Un'area per la gestione delle terapie dei pazienti
- Un'area per la gestione dei pazienti

Ora elenchiamo di seguito i casi d'uso visti nell'immagine:

- ricezione e visualizzazione degli alert per il medico
- Gestione delle terapie dei pazienti
  - Modifica terapia
  - Elimina terapia
  - Crea terapia

- Gestione dei pazienti
  - Visualizza paziente
  - Modifica paziente
  - Elimina paziente

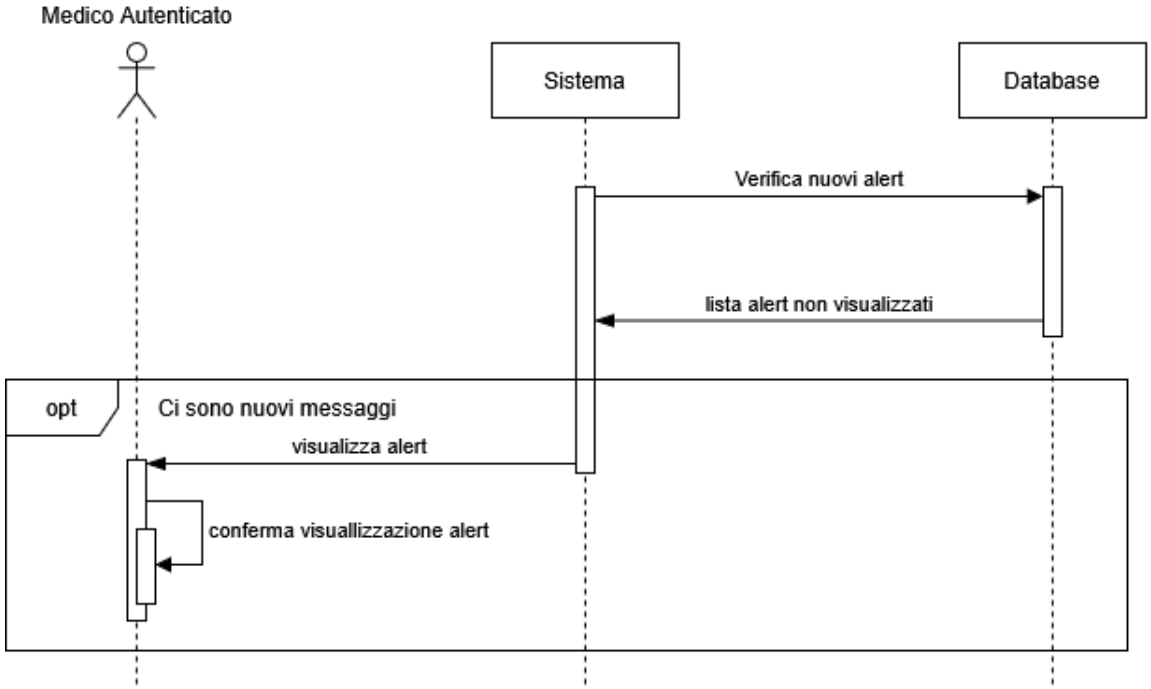
1.2.1 Visualizza alert per il medico

Il medico avrà la lista dei messaggi e degli alert sulla destra della sua homepage; quando avrà letto l'alert o non ne necessiterà più potrà schiacciare il tasto "leggi" per segnarlo come letto e quindi rimuoverlo dalla lista.

Attori: Medico
Precondizioni: Il medico deve essersi autenticato
Passi:

1. Il medico legge l'alert
2. Il medico clicca il pulsante "leggi" per eliminarlo dalla lista

Postcondizioni: l'alert viene rimosso dalla lista del medico



1.2.2 Gestione Terapie

Il medico può decidere di modificare una terapia cliccando il pulsante a forma di matita affianco alla terapia che vuole modificare; in caso volesse eliminarla dovrà schiacciare il pulsante a forma di cestino, con conferma dell'eliminazione, e infine può decidere di crearne una nuova schiacciando il pulsante "Nuova Terapia". Nel caso della creazione e della modifica si aprirà un modal nel quale dovrà inserire i dati richiesti e scegliere il paziente per il quale è stata creata.

Attori: Medico
Precondizioni: Il medico deve essersi autenticato
Passi:

1. Modifica terapia
   (a) Il medico schiaccia il pulsante a forma di matita
   (b) Il medico modifica i dati del modal della terapia
   (c) Il medico schiaccia il pulsante "salva"
2. Aggiungi terapia
   (a) Il medico schiaccia il pulsante "Nuova Terapia"
   (b) Il medico compila i campi richiesti nel modal
   (c) Il medico schiaccia il pulsante "salva"
3. Elimina terapia
   (a) Il medico schiaccia il pulsante a forma di cestino affianco alla terapia che desidera eliminare
   (b) Il medico conferma l'eliminazione della terapia

Postcondizioni: La terapia viene modificata, creata o eliminata in base alle decisioni del medico

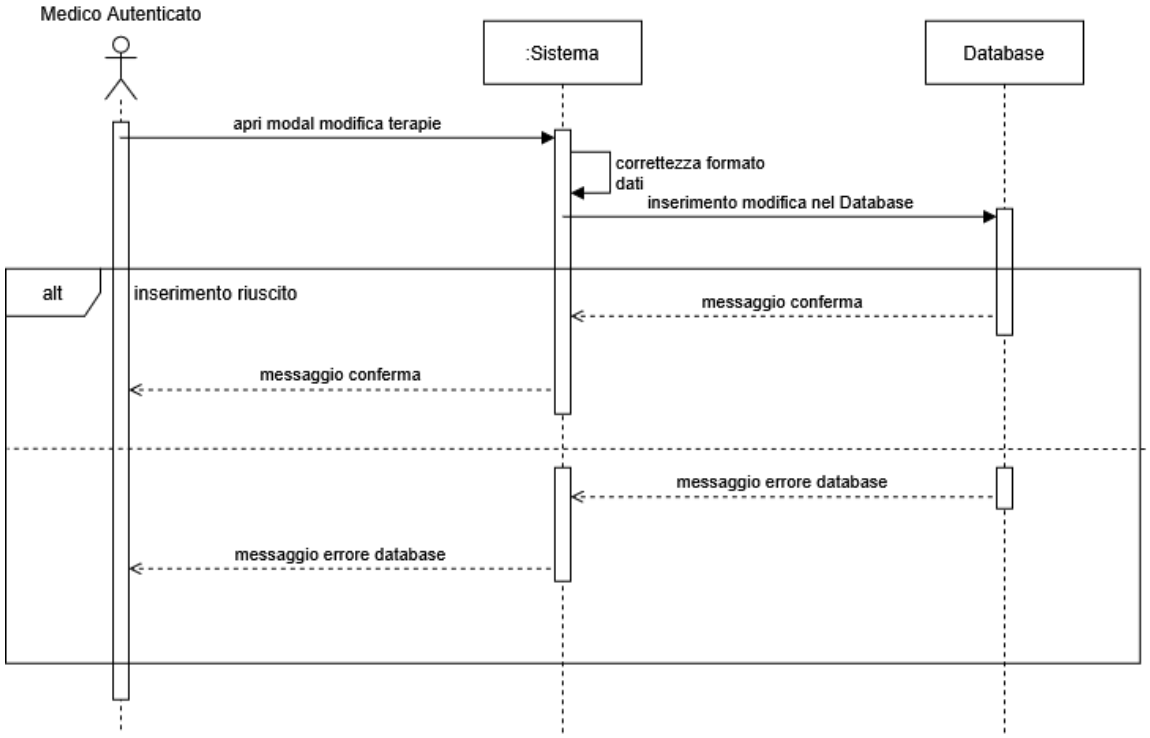


Figure 1.1: Modifica Terapia

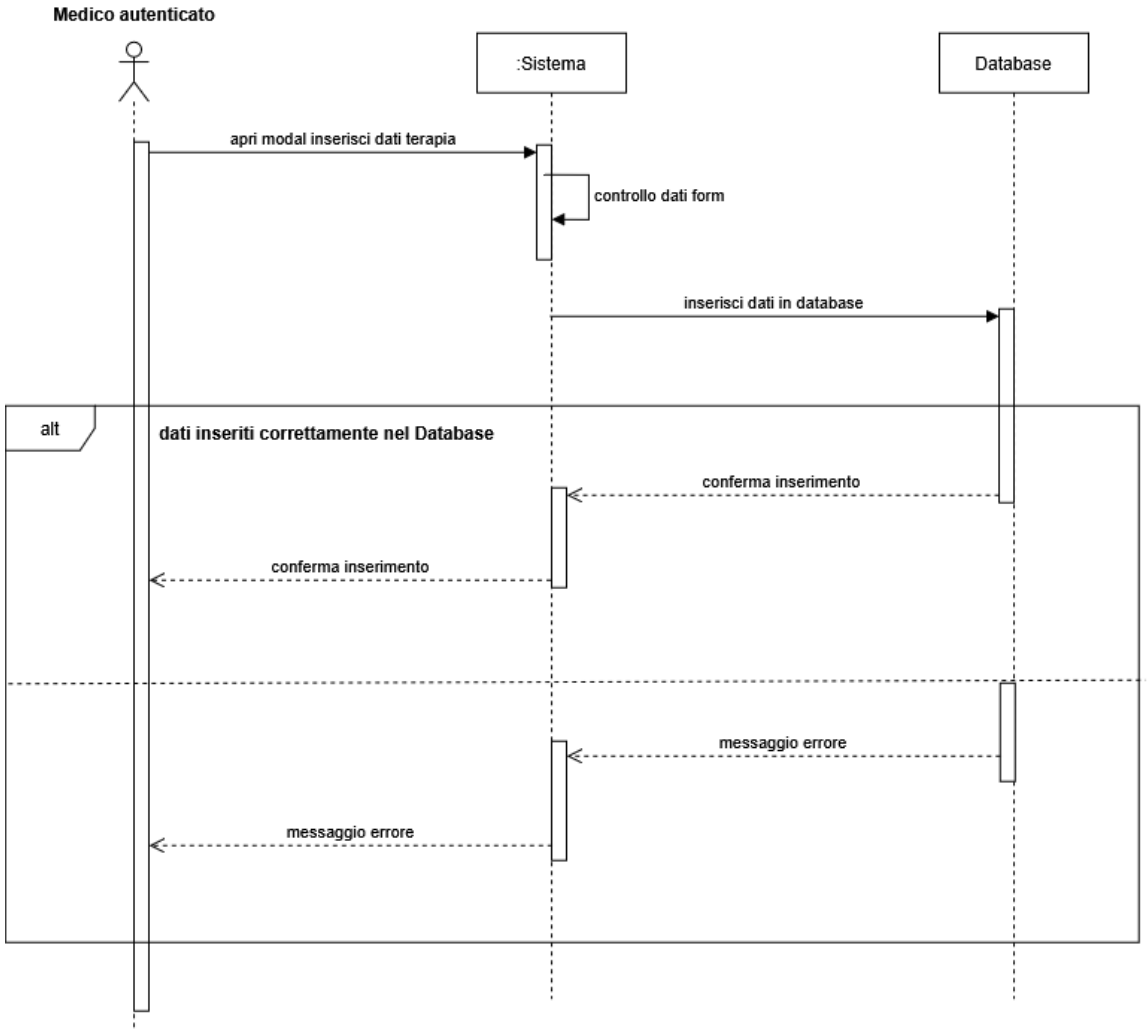


Figure 1.2: Crea Terapia

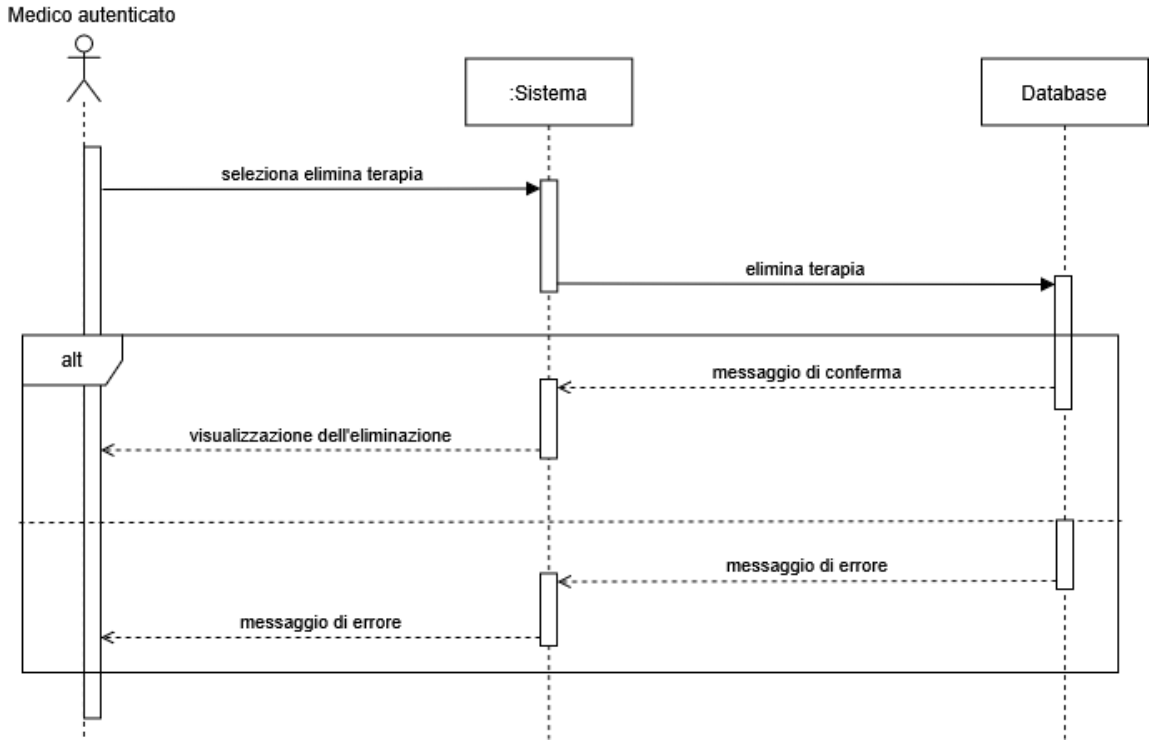


Figure 1.3: Elimina Terapia

1.2.3 Gestione Pazienti

Il medico può decidere di modificare le informazioni di un Paziente cliccando il pulsante a forma di matita affianco al paziente del quale vuole modificare i dati; in caso volesse eliminarlo dovrà schiacciare il pulsante a forma di cestino; nel caso voglia solo visualizzare l'andamento glicemico bisogna che schiacci il pulsante a forma di occhio. Nel caso della creazione si aprirà un modal nel quale dovrà inserire i dati richiesti e scegliere il paziente per il quale è stata creata.

Attori: Medico
Precondizioni: Il medico deve essersi autenticato
Passi:

1. Modifica dati del Paziente
   (a) Il medico schiaccia il pulsante a forma di matita
   (b) Il medico modifica i dati del modal del paziente
   (c) Il medico schiaccia il pulsante "salva"
2. Visualizza dati Paziente
   (a) Il medico schiaccia il pulsante a forma di occhio
3. Elimina Paziente
   (a) Il medico schiaccia il pulsante a forma di cestino affianco al paziente che desidera eliminare
   (b) Il medico conferma l'eliminazione del paziente

Postcondizioni: Il paziente viene modificato o eliminato in base alle decisioni del medico, oppure vengono visualizzati i suoi dati

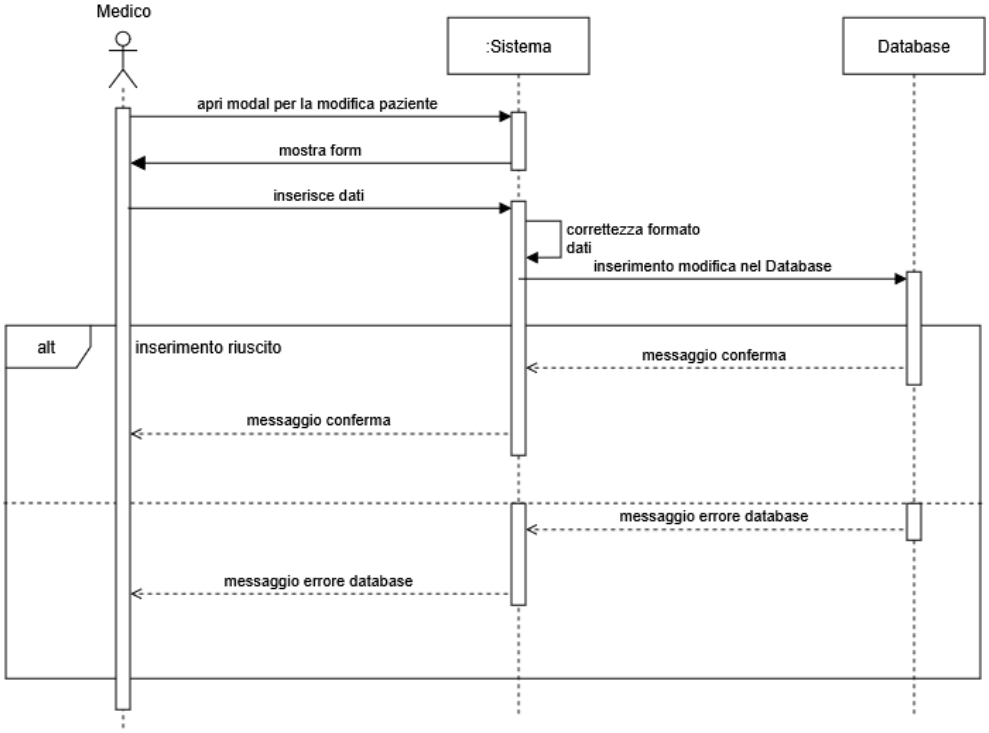


Figure 1.4: Modifica Paziente

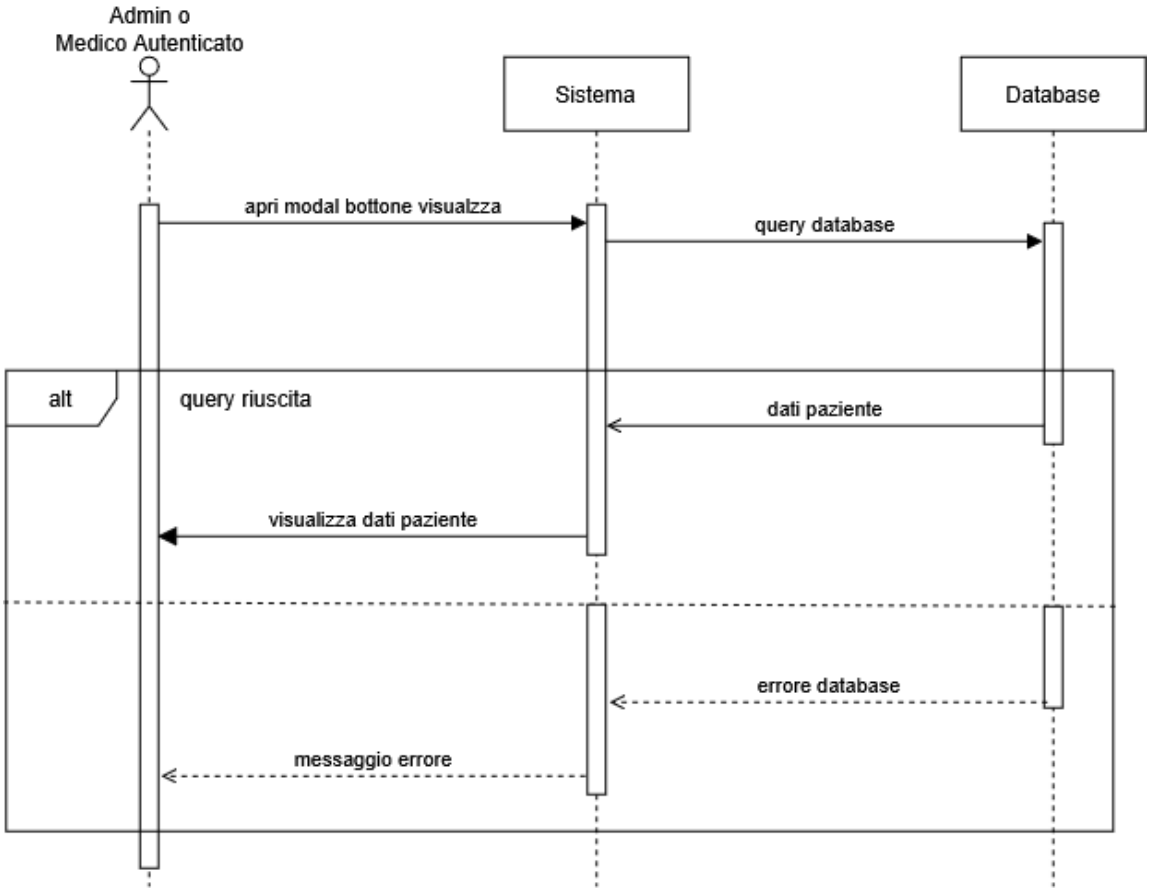


Figure 1.5: Visualizza Paziente

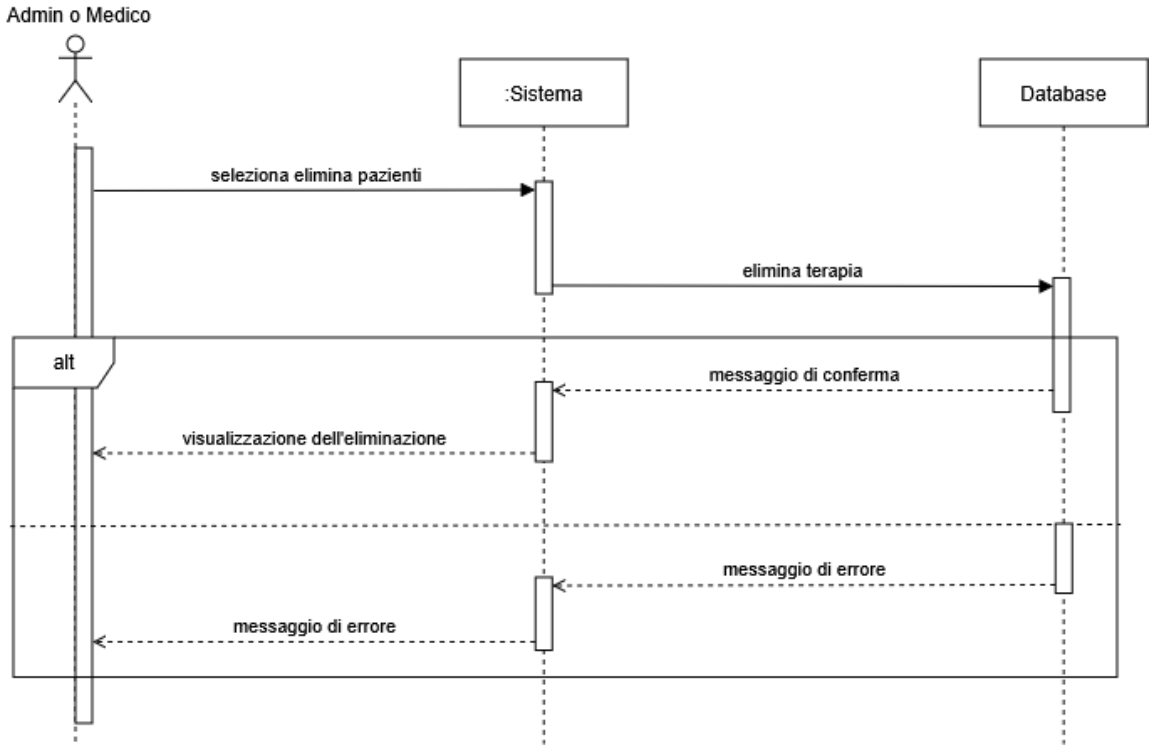
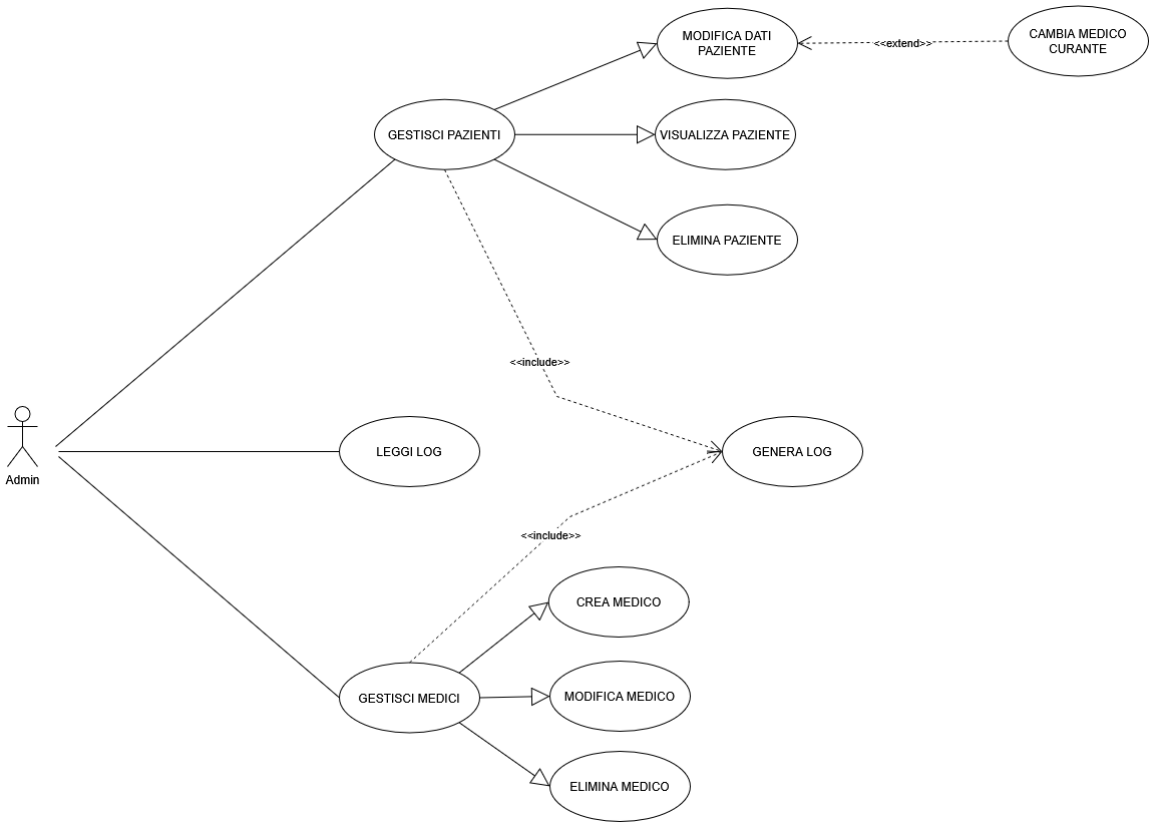


Figure 1.6: Elimina Paziente

1.3 Casi d'uso dell'Admin



Dopo essersi autenticato l'admin verrà reindirizzato alla sua homepage, dove troverà:

- Gestione utenti
  - Gestisci Pazienti
  - Gestisci Medici
- Lista dei log del sistema

Ora elenchiamo di seguito i casi d'uso visti nell'immagine:

- Visualizzazione dei log di sistema
- Gestione dei Pazienti
  - Visualizza paziente
  - Modifica paziente
  - Elimina paziente
  - Crea paziente
- Gestione dei Medici
  - Modifica medico
  - Elimina medico
  - Crea medico

1.3.1 Visualizza log

L'admin può visualizzare i log del sistema per monitorare le attività compiute dagli utenti.

Attori: Admin
Precondizioni: l'Admin deve essersi autenticato
Passi:

1. visualizza i log del sistema

1.3.2 Gestisci Pazienti

L'admin può gestire i pazienti al posto del dottore in caso di necessità come il dottore può modificare, creare, eliminare e visualizzare i pazienti.

Attori: Admin
Precondizioni: l'Admin deve essersi autenticato
Passi:

1. Modifica Paziente
   (a) L'Admin schiaccia il pulsante a forma di matita
   (b) L'Admin modifica i dati del modal del paziente
   (c) L'Admin schiaccia il pulsante "salva"
2. Visualizza Paziente
   (a) L'Admin schiaccia il pulsante a forma di occhio
3. Elimina Paziente
   (a) L'Admin schiaccia il pulsante a forma di cestino affianco al paziente che desidera eliminare
   (b) L'Admin coferma l'eliminazione del paziente
4. Crea Paziente
   (a) l'Admin schiaccia il pulsante "Nuovo Paziente"
   (b) L'Admin compila i campi richiesti nel modal
   (c) L'admin schiaccia il pulsante "Registra"

Postcondizioni: Il paziente viene modificato, eliminato o creato in base alle decisioni dell'Admin, oppure vengono visualizzati i dati del paziente.

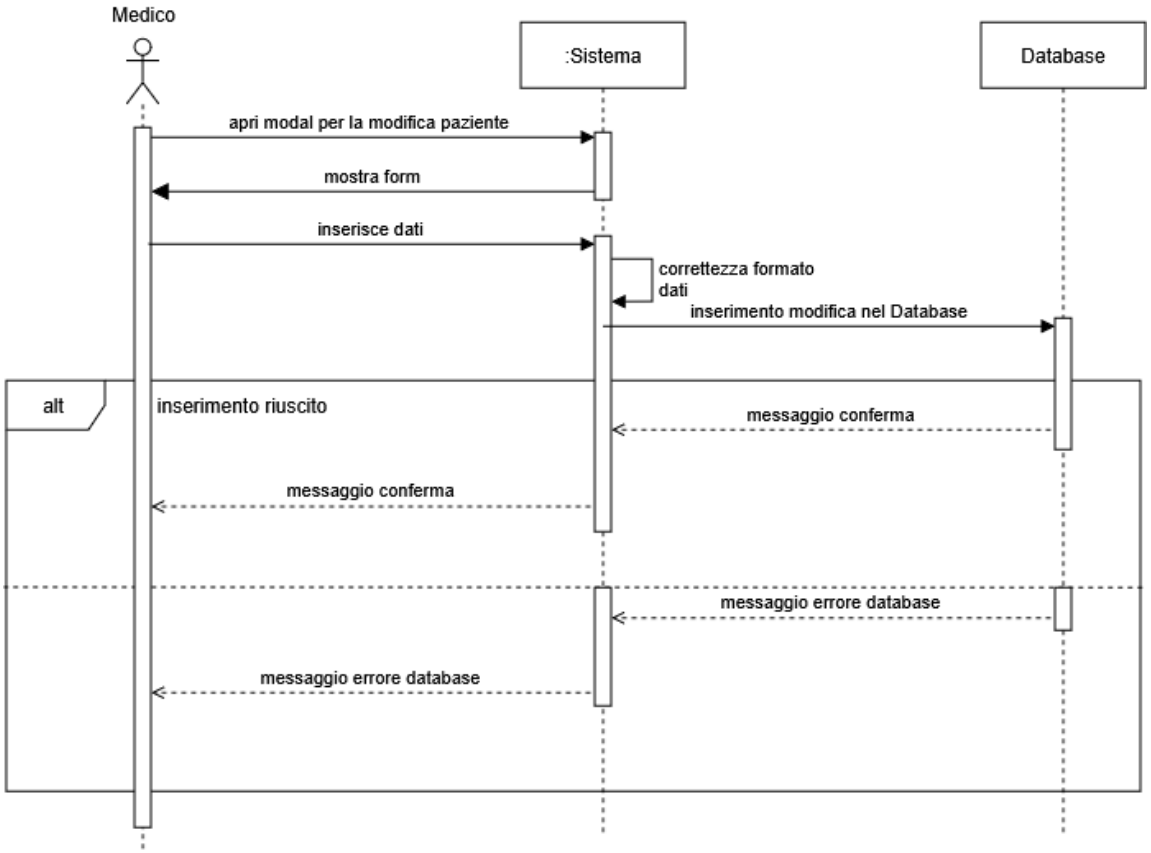


Figure 1.7: Modifica Paziente

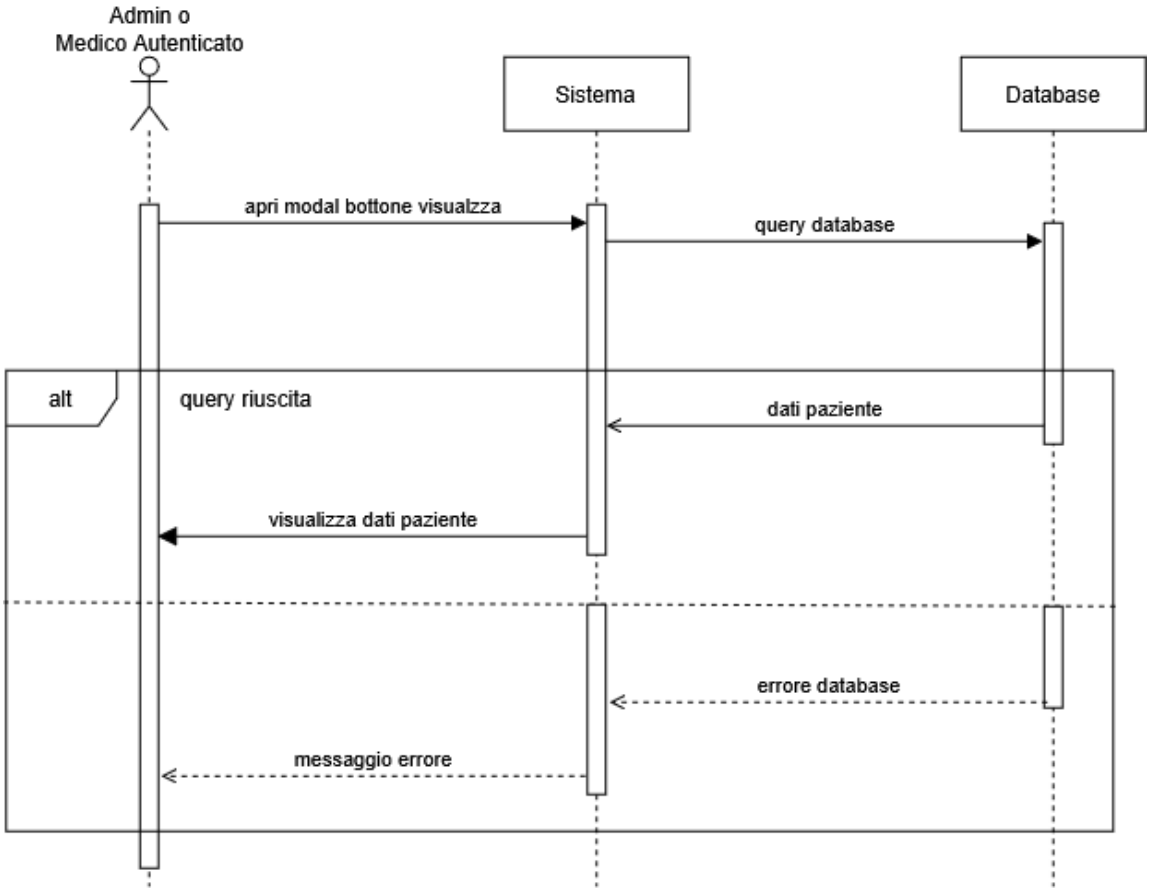


Figure 1.8: Visualizza Paziente

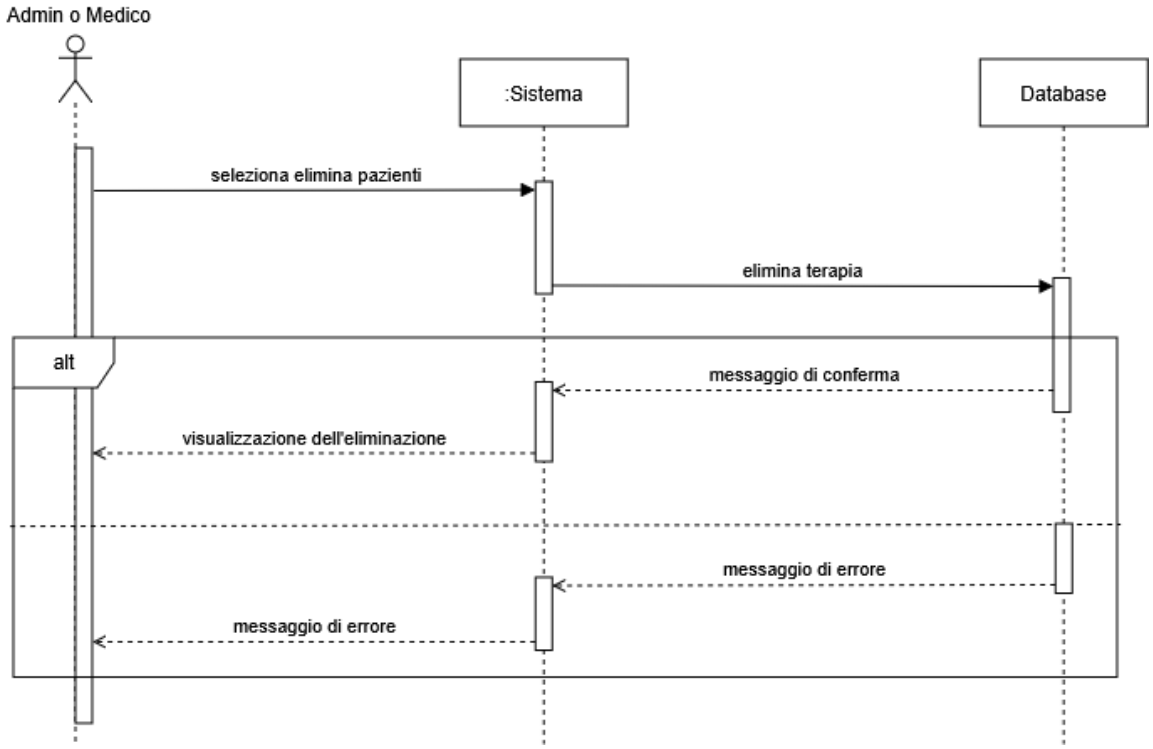


Figure 1.9: Elimina Paziente

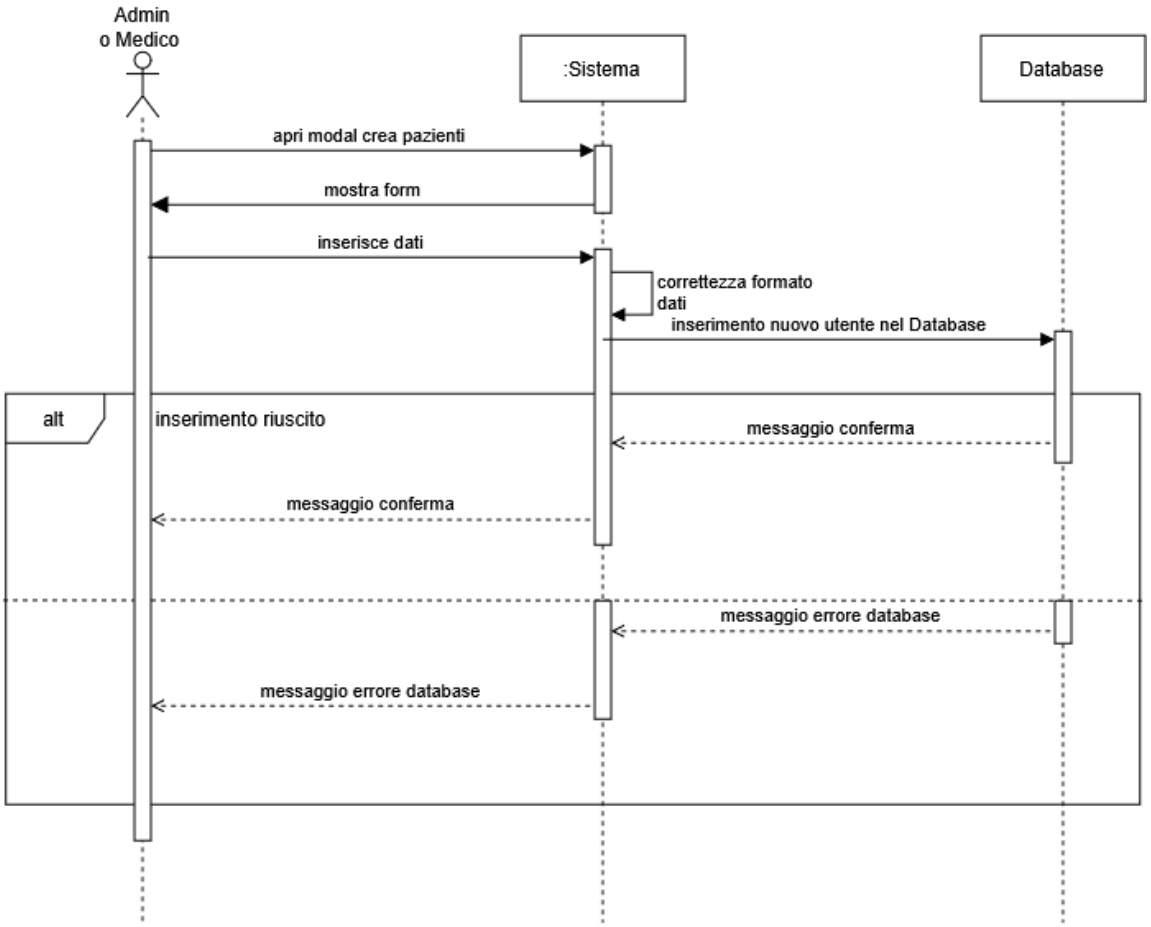


Figure 1.10: Crea Paziente

1.3.3 Gestisci Medici

L'Admin può decidere di modificare le informazioni di un medico cliccando il pulsante a forma di matita affianco al medico del quale vuole modificare i dati; in caso volesse eliminarlo dovrà schiacciare il pulsante a forma di cestino; Nel caso della creazione si aprirà un modal nel quale dovrà inserire i dati richiesti.

Attori: Medico
Precondizioni: Il medico deve essersi autenticato
Passi:

1. Modifica Medico
   (a) L'Admin schiaccia il pulsante a forma di matita
   (b) L'Admin modifica i dati del modal del medico
   (c) L'Admin schiaccia il pulsante "salva"
2. Elimina Medico
   (a) L'Admin schiaccia il pulsante a forma di cestino affianco al medico che desidera eliminare
   (b) L'Admin conferma l'eliminazione del medico
3. Crea Medico
   (a) l'Admin schiaccia il pulsante "Nuovo Medico"
   (b) L'Admin compila i campi richiesti nel modal
   (c) L'admin schiaccia il pulsante "Registra"

Postcondizioni: Il Medico viene modificato, eliminato o creato in base alle decisioni dell'Admin.

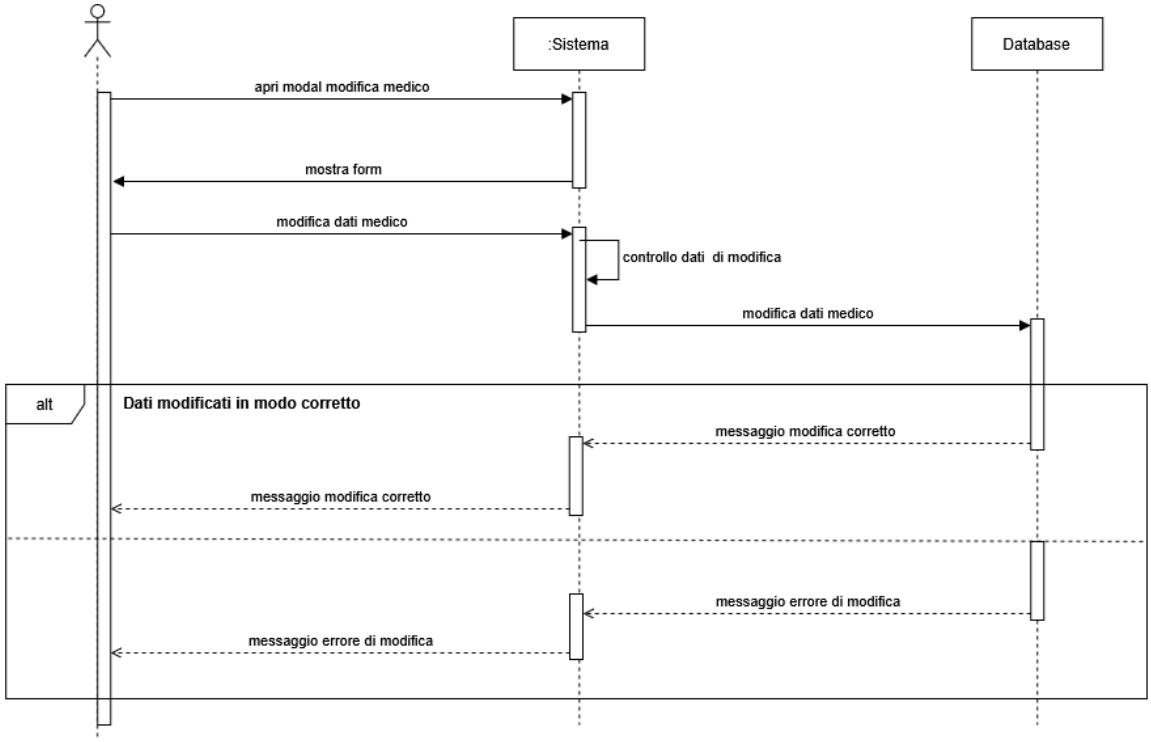


Figure 1.11: Modifica Medico

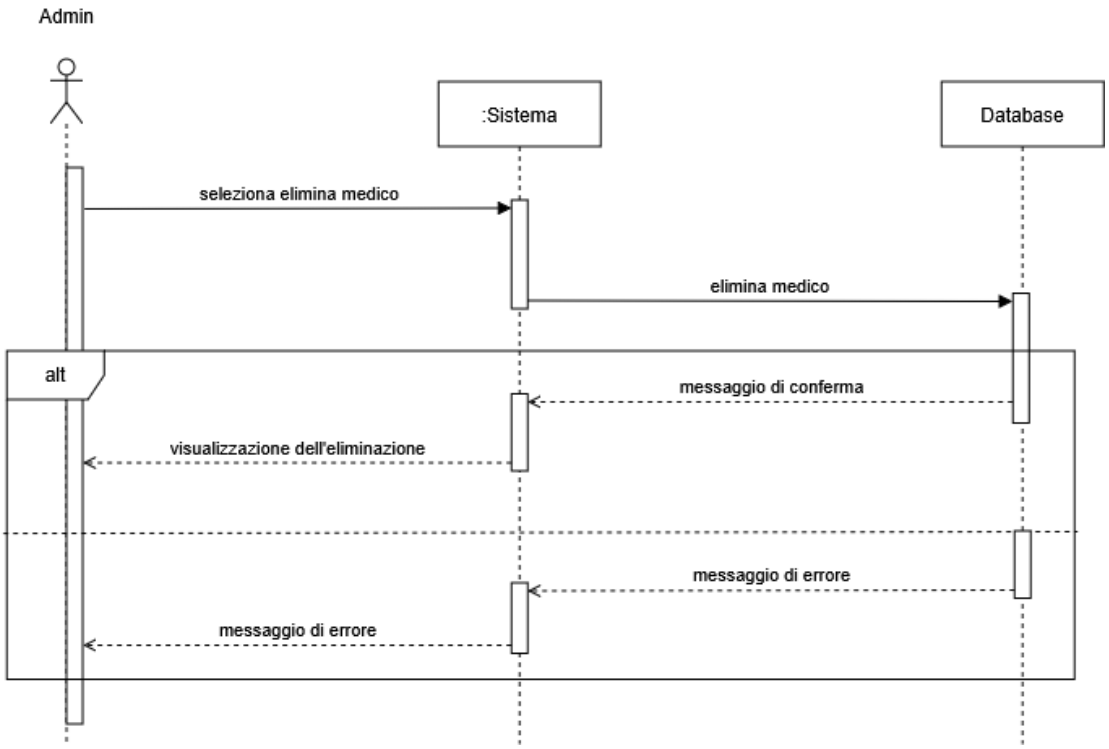


Figure 1.12: Elimina Medico

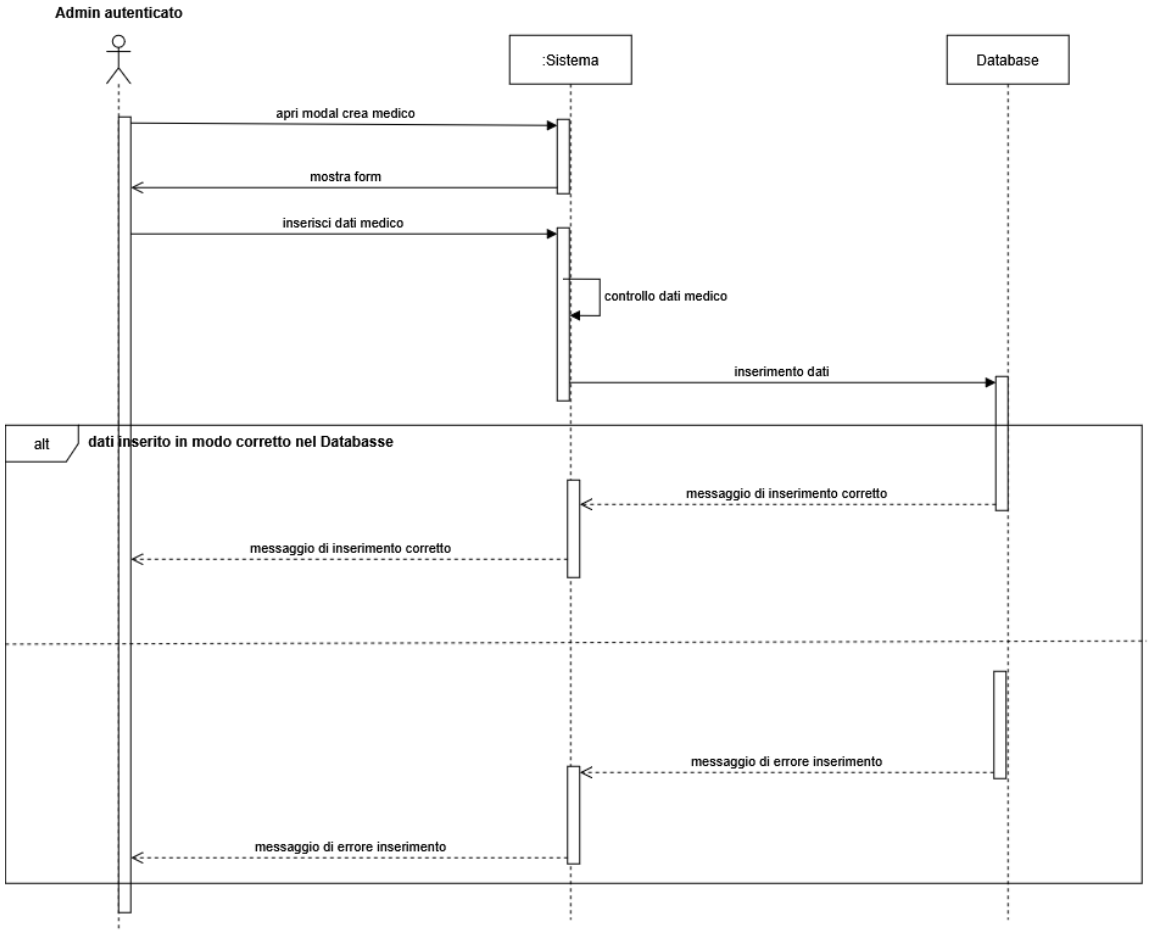


Figure 1.13: Crea Medico

Chapter 2

Diagrammi di attività

Andiamo di seguito a mostrare i diagrammi di attività di ciascuna tipologia di utente.

2.1 Diagramma di autenticazione

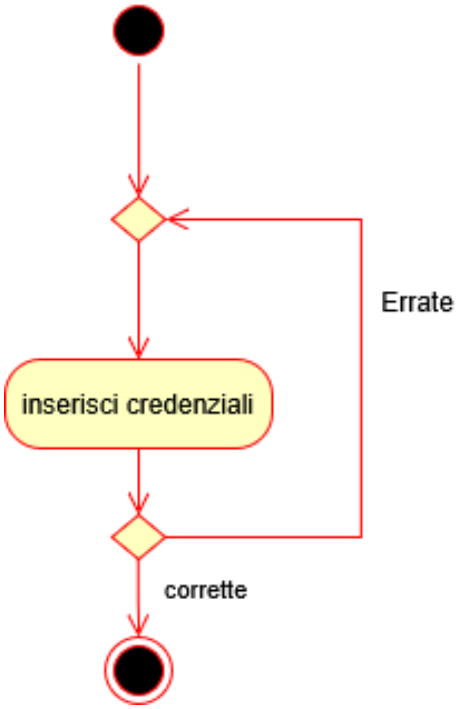
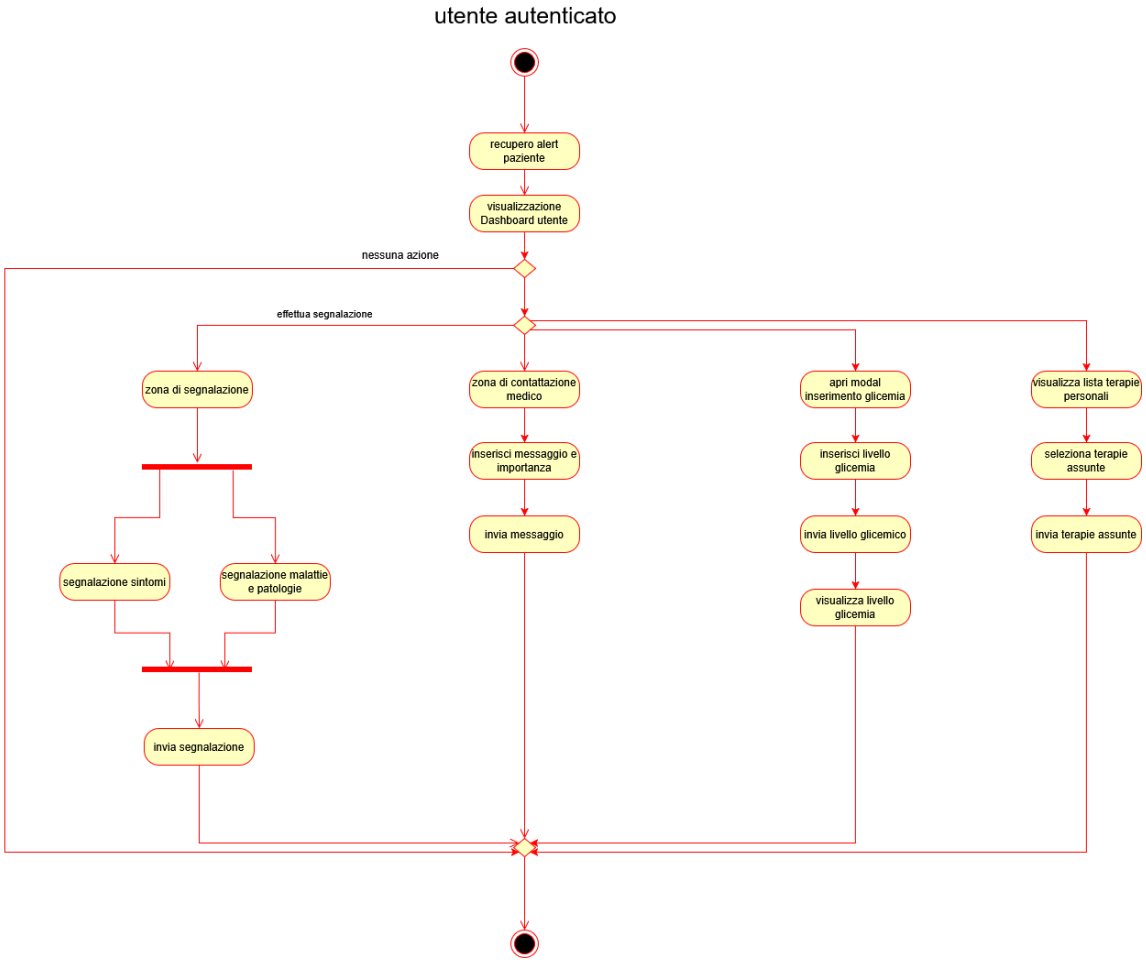


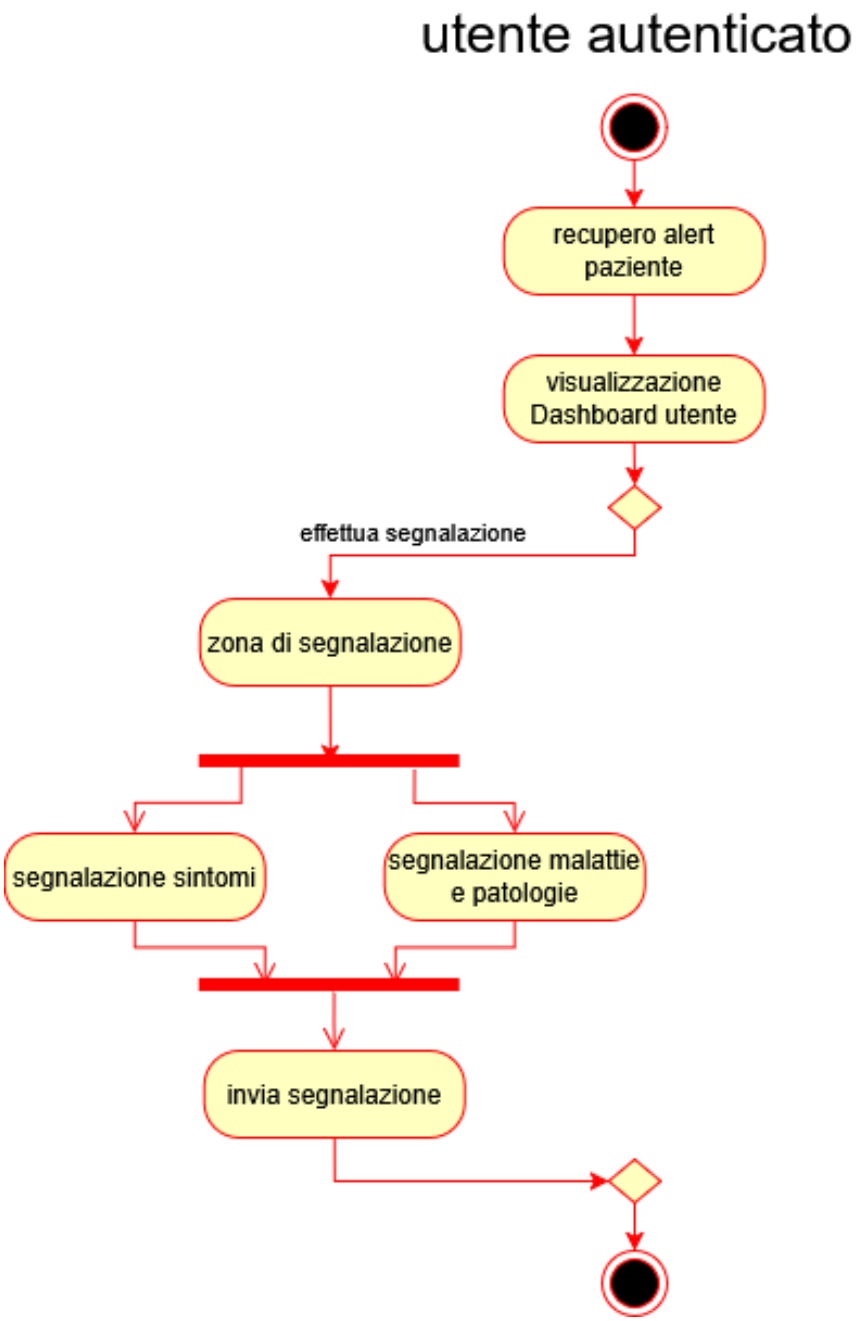
Figure 2.1: Diagramma dell'autenticazione di un utente

Per i successivi diagrammi di attività presupponiamo che l'utente abbia già effettuato l'autenticazione. I diagrammi verranno visualizzati prima completi con per ogni utente e poi divisi per avere una più nitida visualizzazione delle varie sezioni. Le immagini delle diverse sezioni possono avere una struttura diversa da quella nella foto principale a causa del ridimensionamento per una migliore visualizzazione.

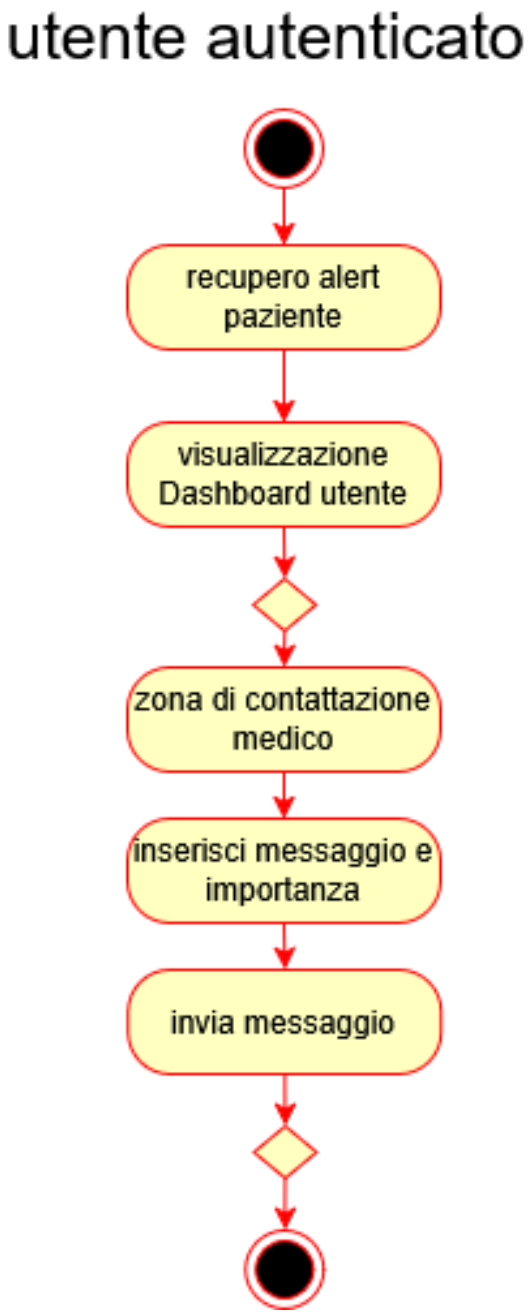
2.2 Diagramma del paziente



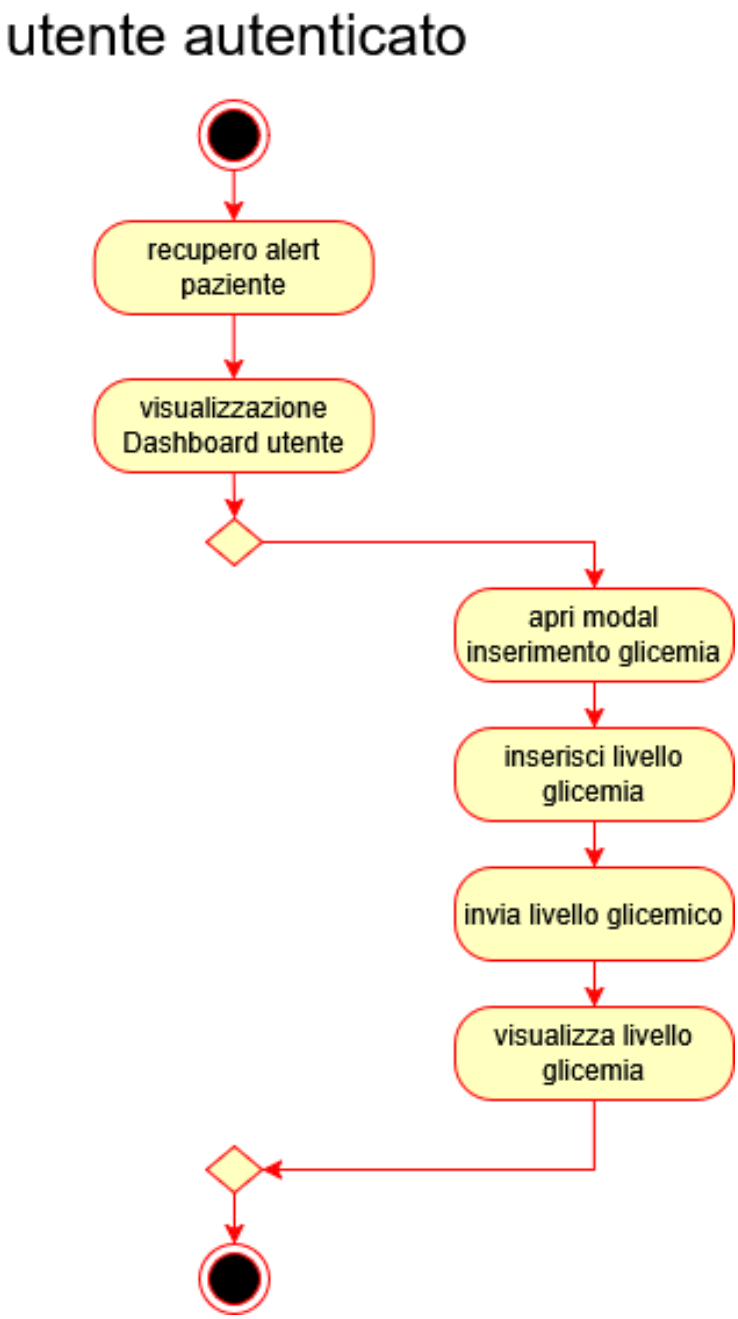
2.2.1 Diagramma per la segnalazione di patologie, malattie e/o sintomi



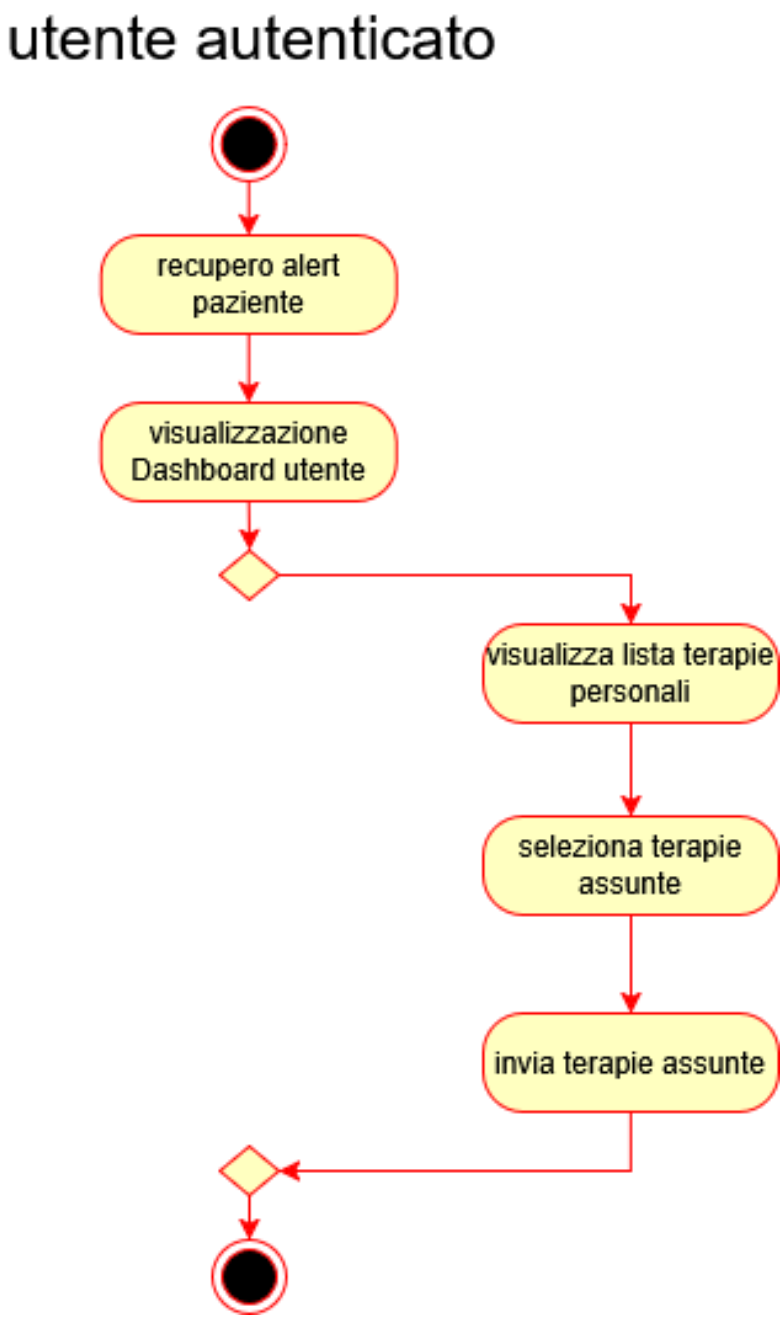
2.2.2 Diagramma di come un paziente comunica con il medico



2.2.3 Diagramma per l'inserimento della glicemia



2.2.4 Diagramma per la registrazione giornaliera dei medicinali



2.3 Diagramma del medico

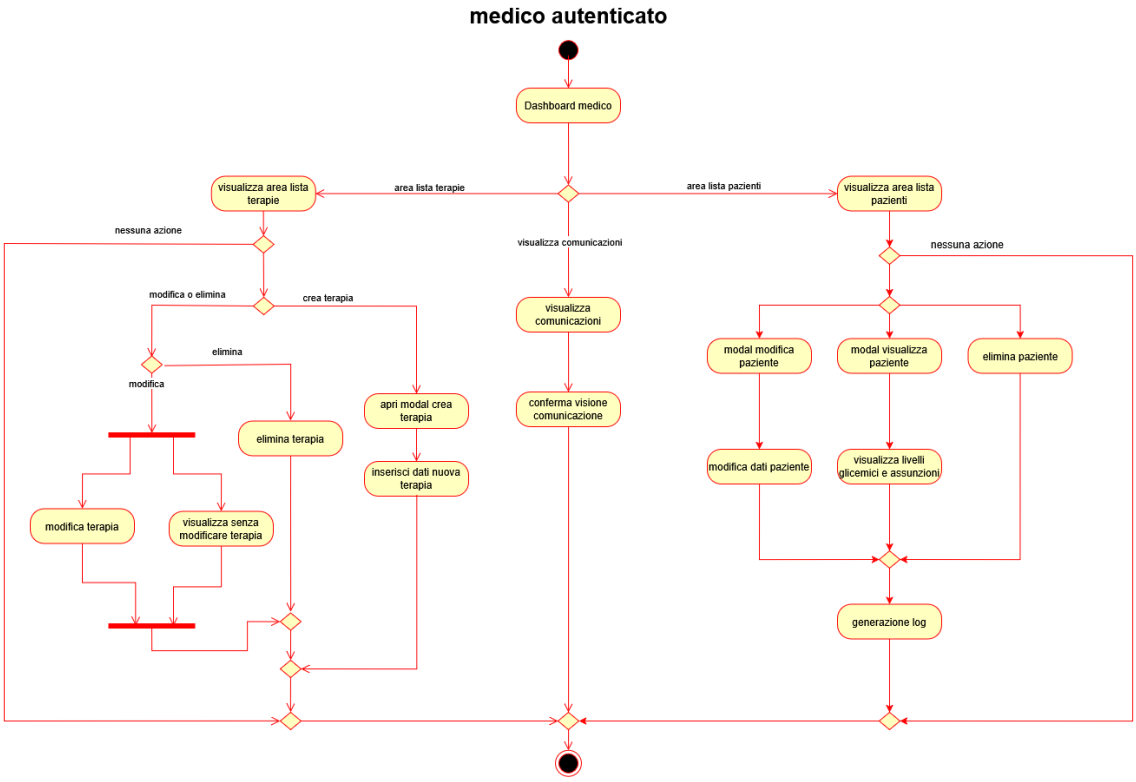
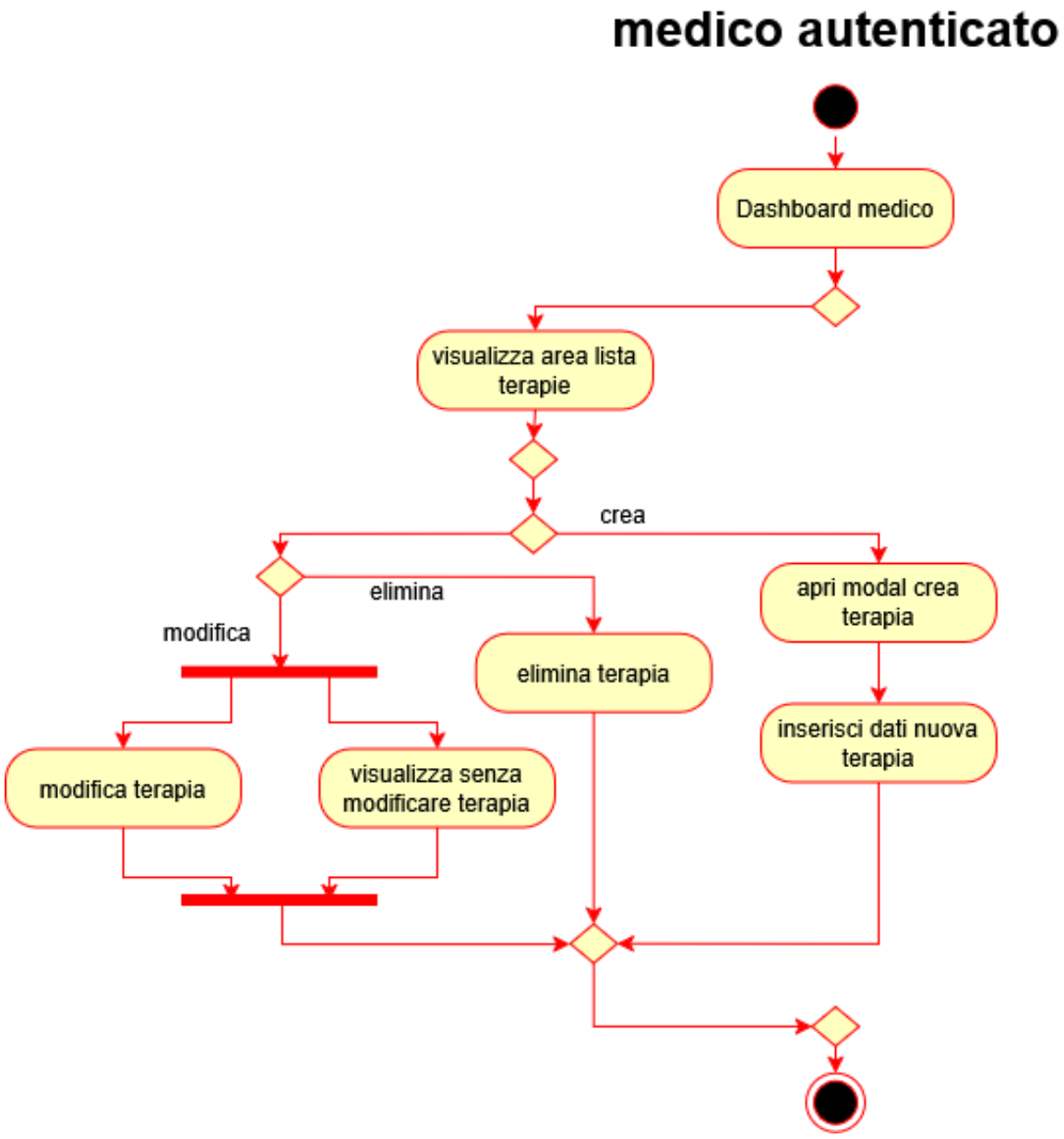
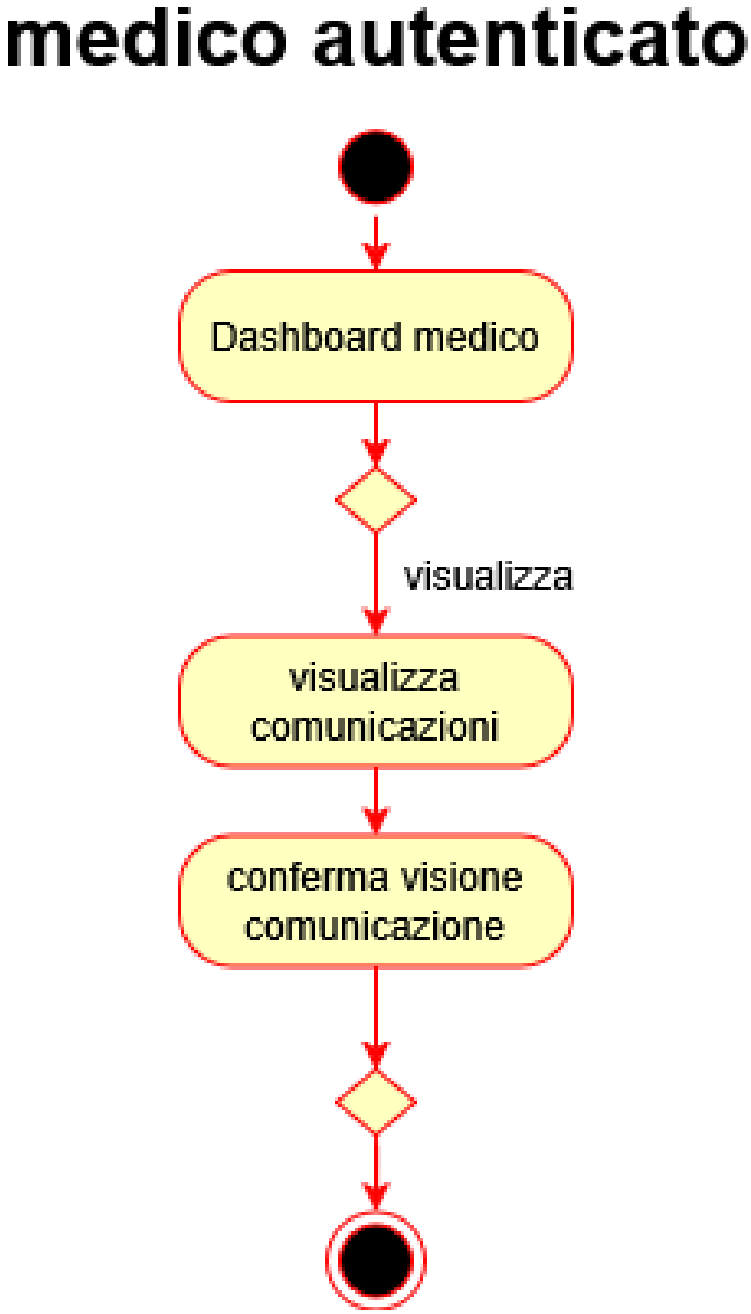


Figure 2.2: Diagramma completo del medico

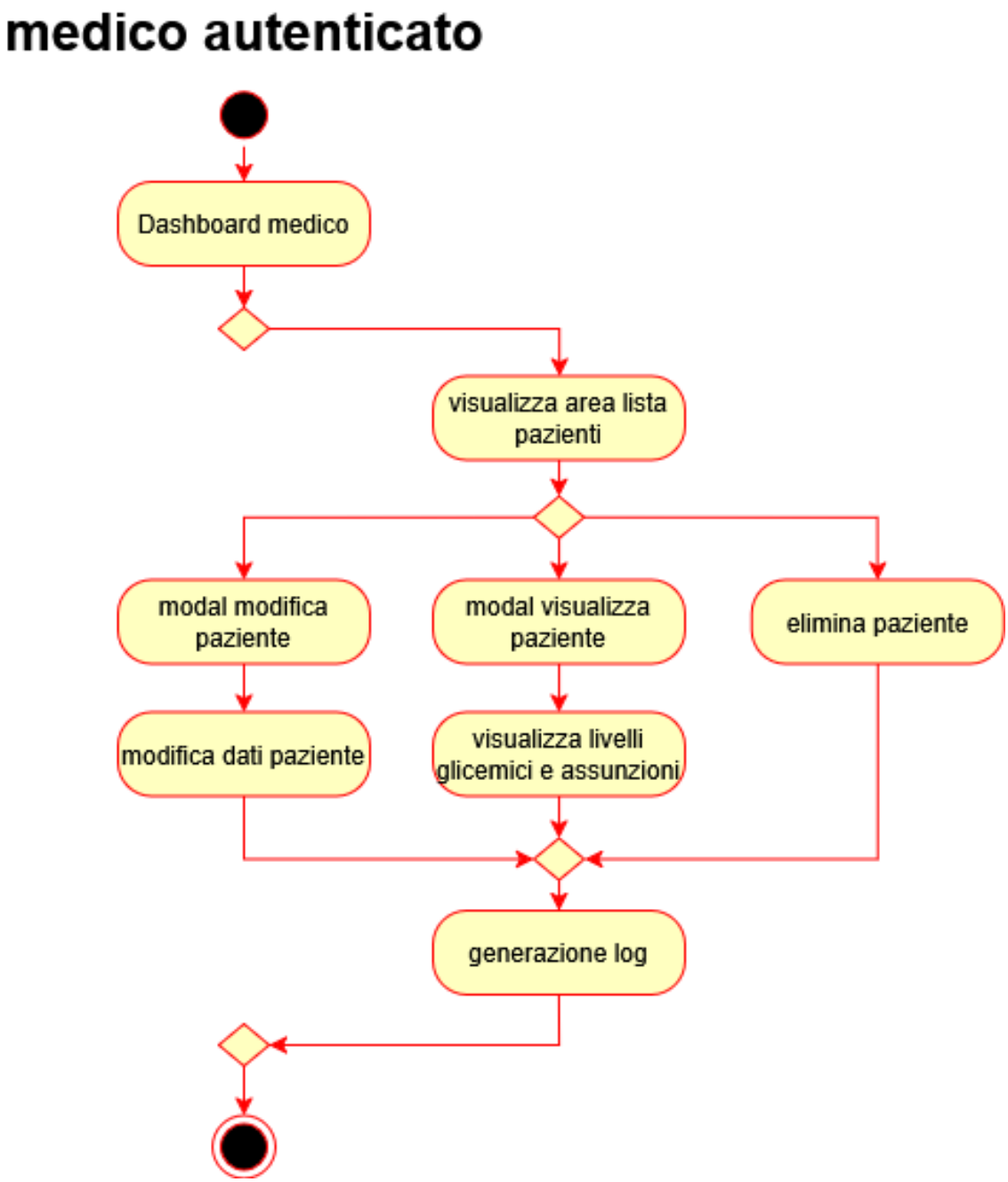
2.3.1 Diagramma di gestione delle terapie



2.3.2 Diagramma di visualizzazione degli alert del medico



2.3.3 Diagramma di gestione dei pazienti



2.4 Diagramma dell'admin

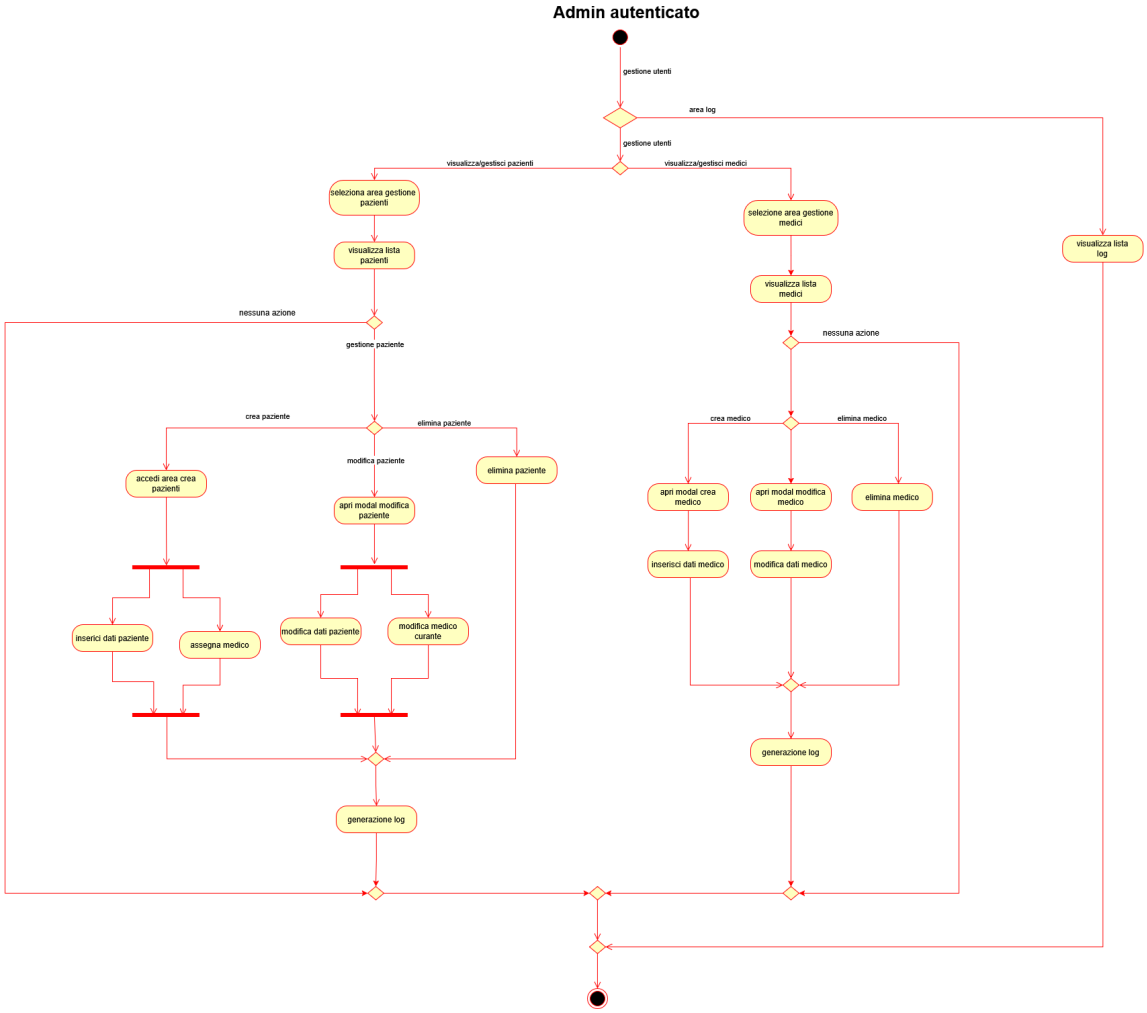
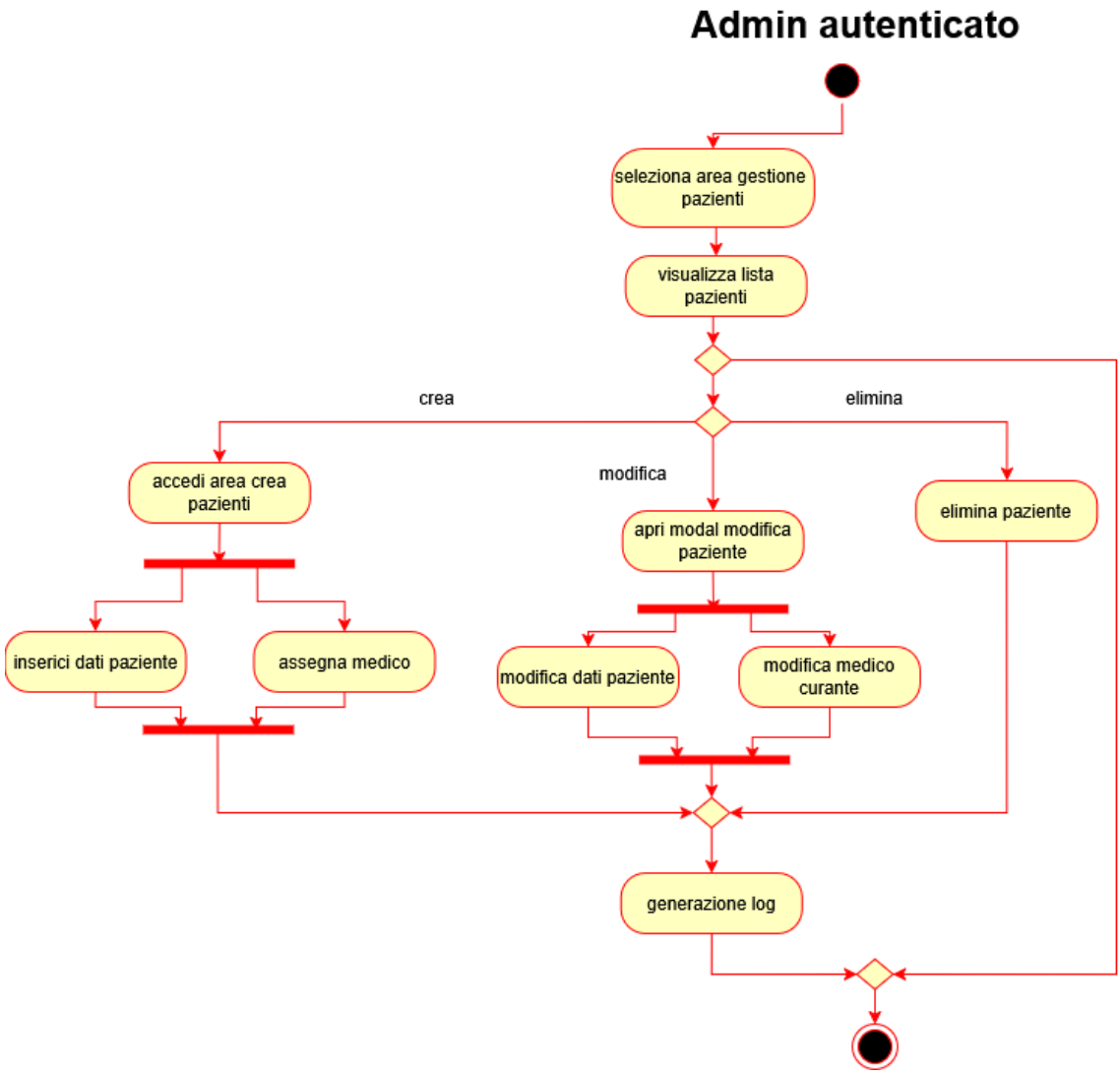
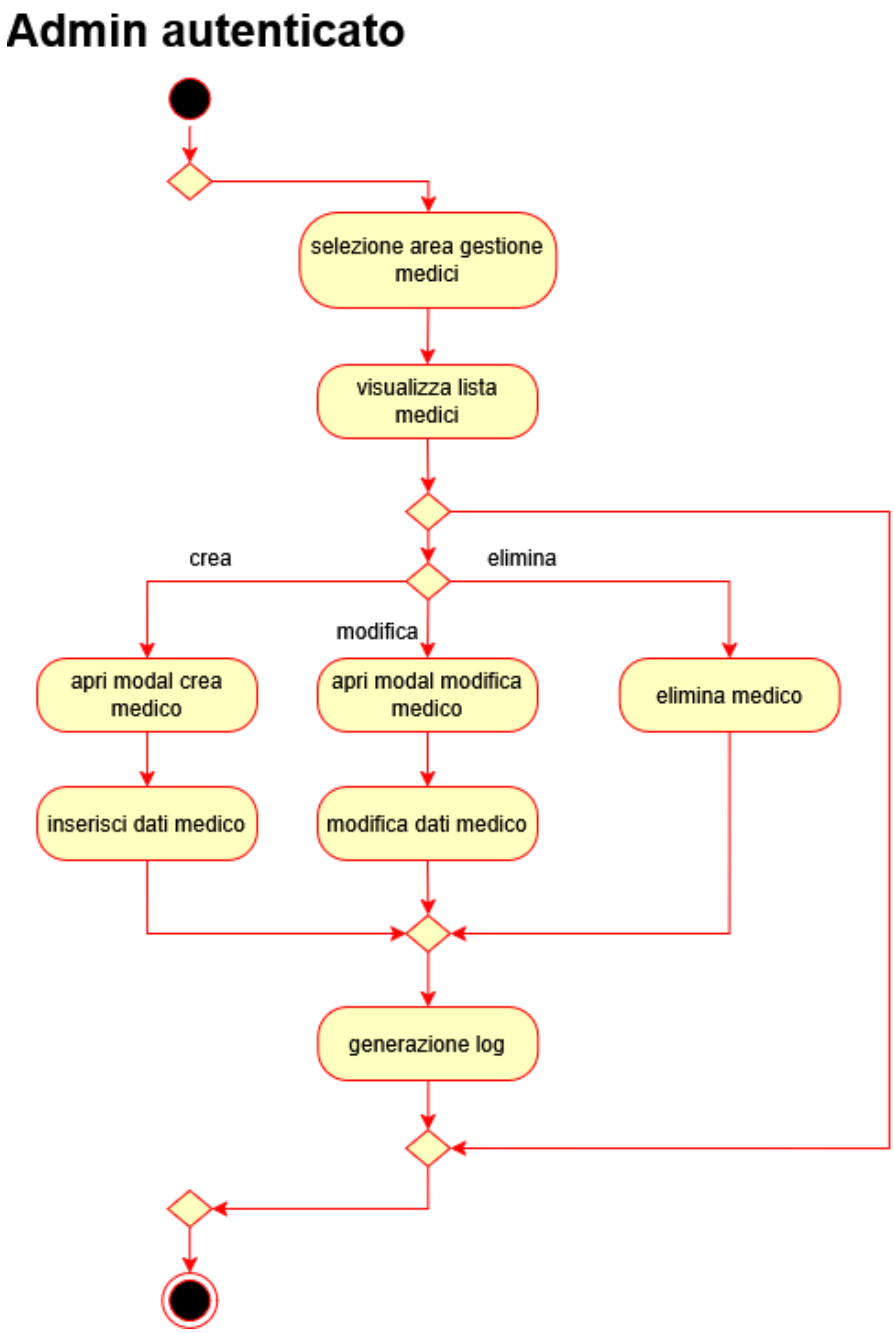


Figure 2.3: Diagramma completo dell'admin

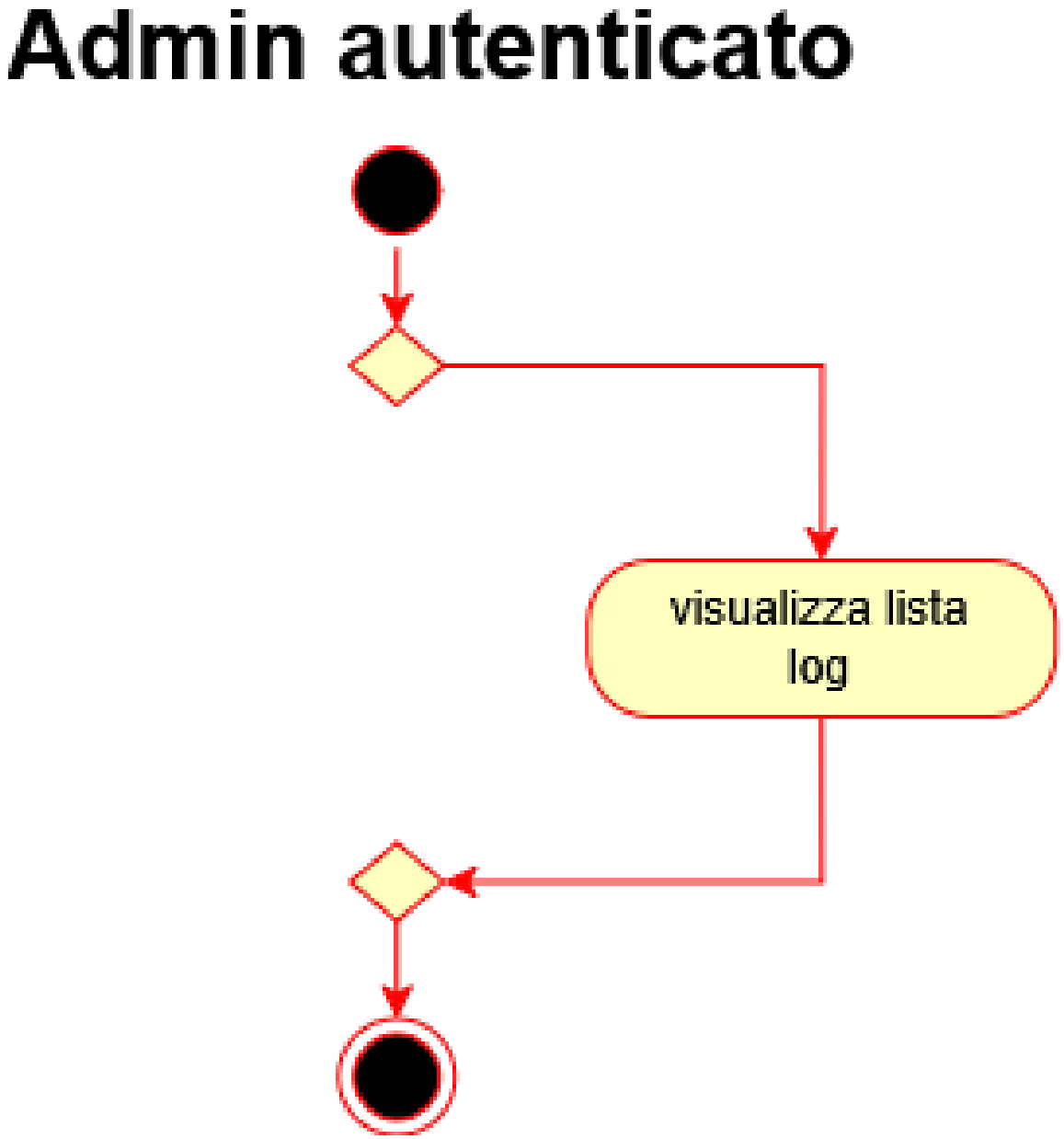
2.4.1 Diagramma gestione dei pazienti



2.4.2 Diagramma della gestione dei medici



2.4.3 Diagramma visualizzazione dei log



Chapter 3

Diagrammi UML

3.1 Diagramma delle classi - Package Entity

Il package `entity` definisce la rappresentazione a oggetti dello schema dati. Di seguito è mostrato il diagramma UML delle entità JPA, che costituiscono il nucleo del sistema, con particolare enfasi sulla mappatura delle relazioni e sulla definizione degli attributi.

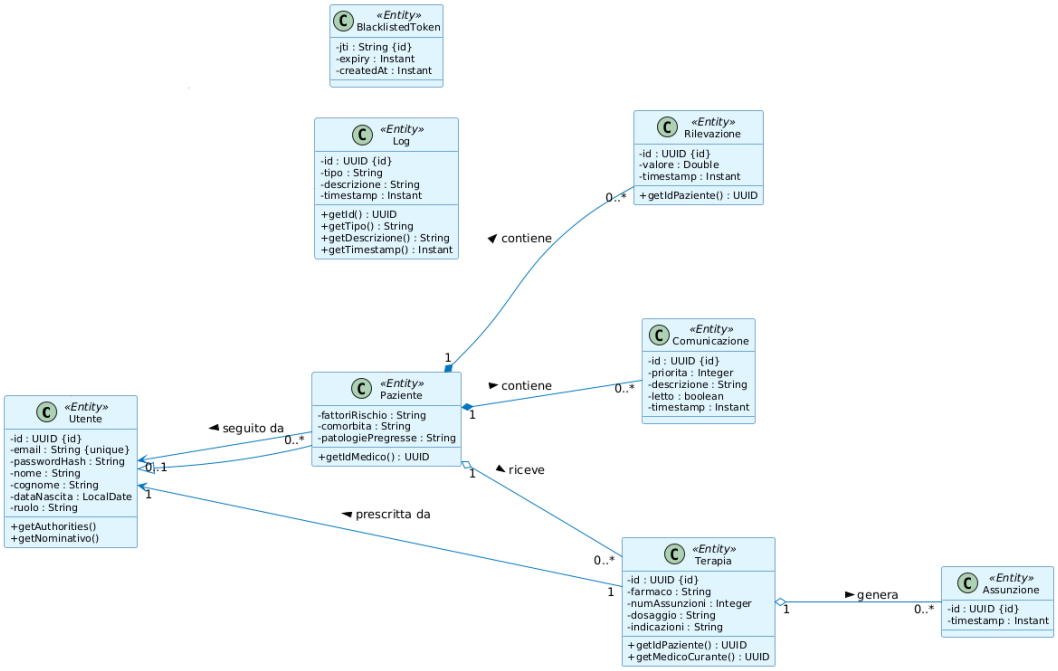


Figure 3.1: Diagramma delle classi - Package Entity

3.1.1 Entità principali

Utente:

- Classe base per tutti gli utenti del sistema
- Utilizza strategia di ereditarietà JOINED (tabelle separate per sottoclassi)
- Contiene dati anagrafici comuni: email, password, nome, cognome, data nascita
- Campo `ruolo` per distinguere: ROLE_PAZIENTE, ROLE_MEDICO, ROLE_ADMIN

Paziente (estende Utente):

- Aggiunge informazioni cliniche: fattoriRischio, comorbita, patologiePregresse
- Associato a un medico curante (relazione ManyToOne con Utente)
- Possiede collezioni di: Rilevazione, Comunicazione, Terapia, Assunzione

Terapia:

- Rappresenta una prescrizione farmacologica
- Attributi: farmaco, numAssunzioni, dosaggio, indicazioni
- Prescritta da un medico (ManyToOne con Utente)
- Associata a un paziente (ManyToOne con Paziente)
- Genera assunzioni registrate dal paziente (OneToMany con Assunzione)

Assunzione:

- Registra quando il paziente assume un farmaco
- Associata a una Terapia specifica (ManyToOne)
- Include timestamp per tracciamento temporale

Rilevazione:

- Memorizza una misurazione della glicemia
- Attributi: valore (Double in mg/dL), timestamp
- Associata a un paziente (ManyToOne)
- Range normale: 80-130 mg/dL (80-180 mg/dL dopo i pasti)

Comunicazione:

- Messaggi e alert tra paziente e medico
- Livello di priorità: 1=bassa, 2=media, 3=alta
- Flag `letto` per gestire stato lettura

- Può essere generata automaticamente dal sistema (alert)

Log:

- Registro eventi di sistema per audit
- Attributi: tipo, descrizione, timestamp
- Entità standalone senza relazioni

BlacklistedToken:

- Gestione token JWT invalidati (logout)
- Chiave primaria: jti (JWT ID univoco)
- Include expiry per cleanup automatico token scaduti

3.2 Diagrammi di sequenza UML

3.2.1 Autenticazione (Sign-In)

Il processo di autenticazione permette agli utenti di accedere al sistema utilizzando email e password. L'autenticazione è gestita tramite JWT (JSON Web Token).

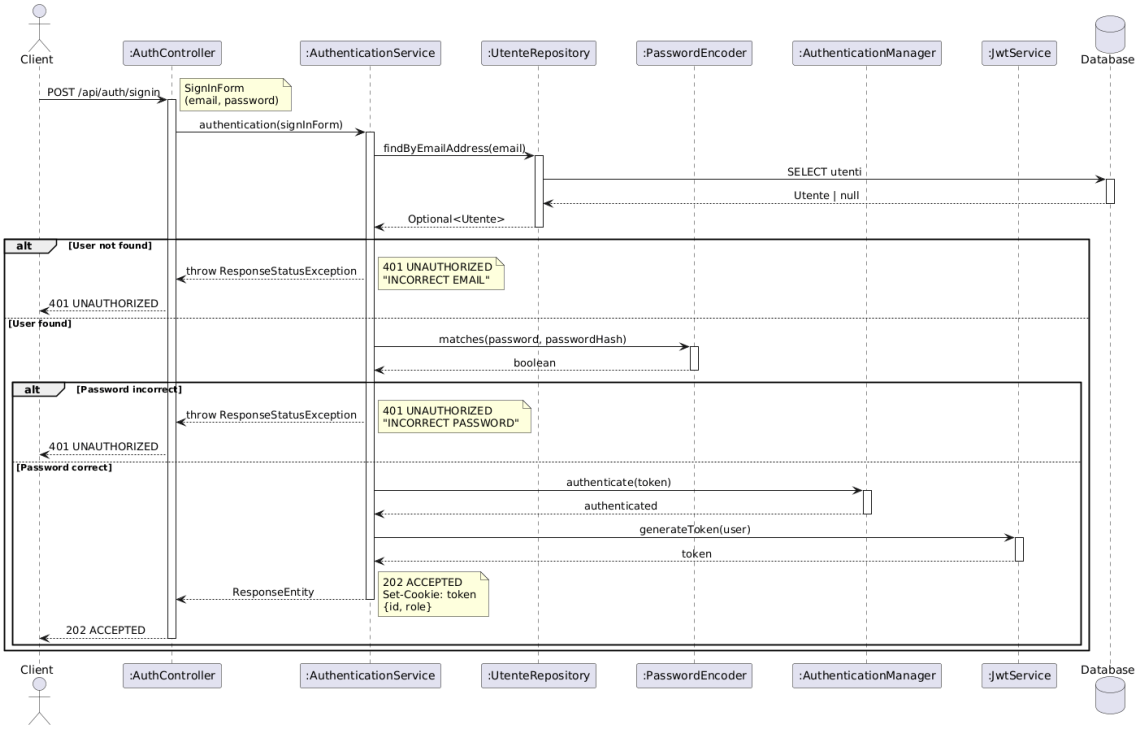


Figure 3.2: Diagramma di sequenza UML - Autenticazione

Flusso principale:

1. L'utente invia credenziali (email, password) via POST /api/auth/signin
2. AuthController delega ad AuthenticationService
3. Il sistema verifica l'esistenza dell'email tramite UtenteRepository
4. PasswordEncoder valida la password tramite hash BCrypt
5. AuthenticationManager autentica l'utente in Spring Security
6. JwtService genera un token JWT firmato
7. Il token viene inviato al client in un cookie HttpOnly sicuro
8. Response 202 ACCEPTED con id utente e ruolo

Scenari alternativi:

- Email non esistente → 401 Unauthorized "INCORRECT EMAIL"
- Password errata → 401 Unauthorized "INCORRECT PASSWORD"

3.2.2 Registrazione utente

La registrazione permette la creazione di nuovi account per pazienti, medici o amministratori, con validazione univocità email e hash sicuro della password.

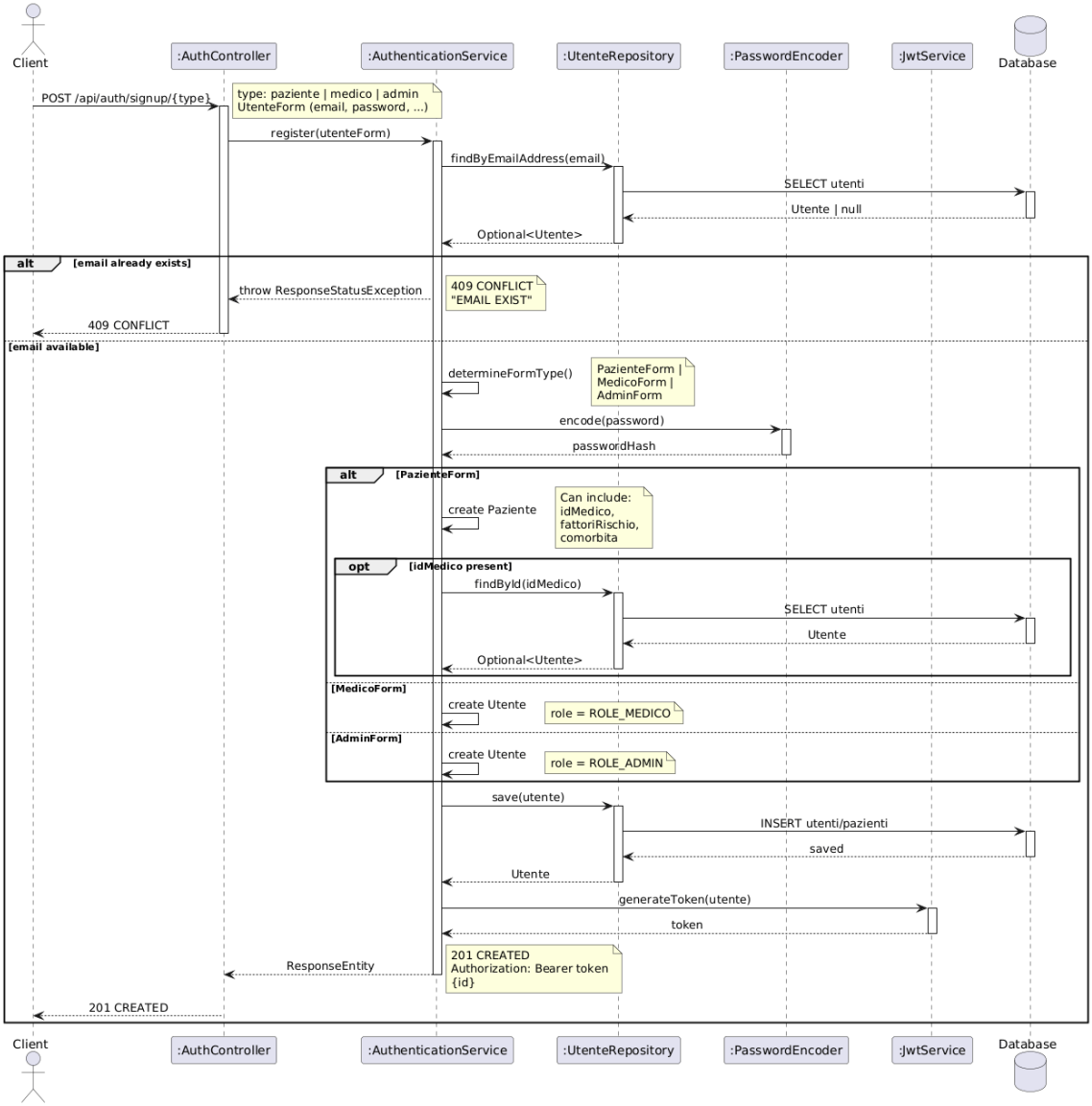


Figure 3.3: Diagramma di sequenza UML - Registrazione

Caratteristiche:

- Endpoint differenziati: /api/auth/signup/{paziente|medico|admin}
- Verifica univocità email nel database
- Hash della password con BCrypt (PasswordEncoder)
- Tipologia utente determinata dal Form ricevuto
- Per i pazienti: possibilità di associare medico curante (idMedico opzionale)
- Generazione token JWT post-registrazione
- Response 201 CREATED con token in header Authorization

Gestione errori:

- Email già esistente → 409 CONFLICT "EMAIL EXIST"
- Dati form invalidi → 400 BAD REQUEST con dettagli validazione

3.2.3 Creazione terapia

Il medico può prescrivere una terapia farmacologica a un paziente specificando farmaco, dosaggio e frequenza di assunzione.

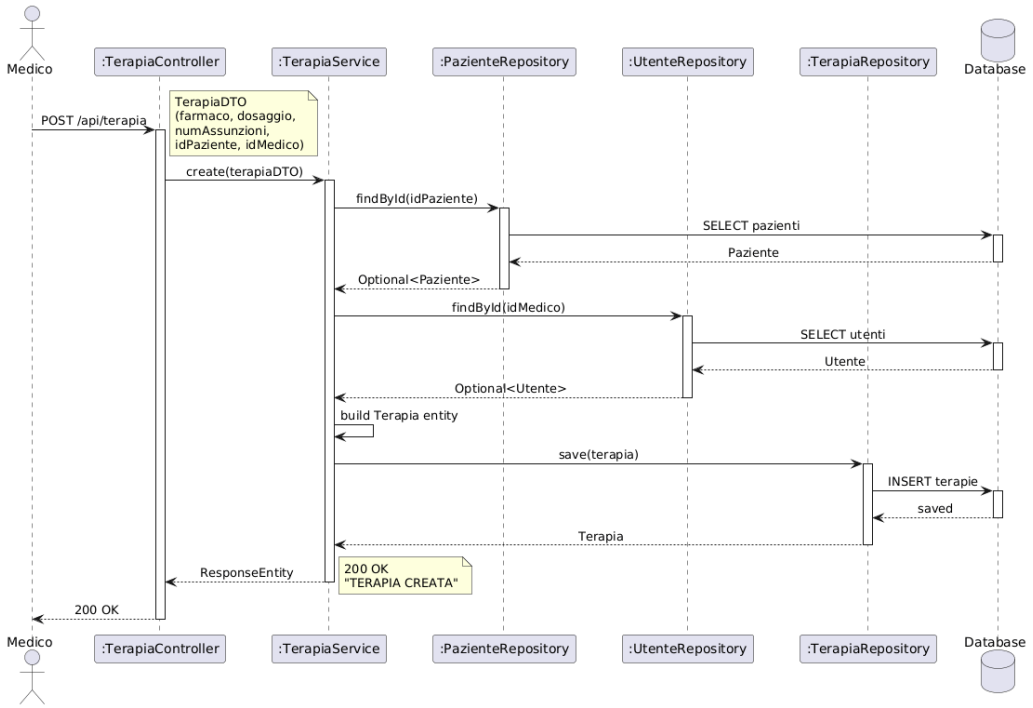


Figure 3.4: Diagramma di sequenza UML - Creazione terapia

Dati della terapia (TerapiaDTO):

- `farmaco`: Nome del farmaco (String)
- `dosaggio`: Dosaggio prescritto (String, es. "500mg")
- `numAssunzioni`: Assunzioni giornaliere (Integer)
- `indicazioni`: Note terapeutiche opzionali (String)
- `idPaziente`: UUID del paziente
- `idMedico`: UUID del medico prescrittore

Flusso:

1. Medico invia POST /api/terapia con TerapiaDTO
2. TerapiaService recupera Paziente e Medico dai repository
3. Crea entità Terapia con relazioni ManyToOne impostate
4. Salva nel database tramite TerapiaRepository
5. Response 200 OK "TERAPIA CREATA"

3.2.4 Inserimento rilevazione con alert automatico

Quando il paziente inserisce una rilevazione glicemica, il sistema controlla automaticamente se i valori sono fuori range normale e genera alert per il medico in tempo reale.

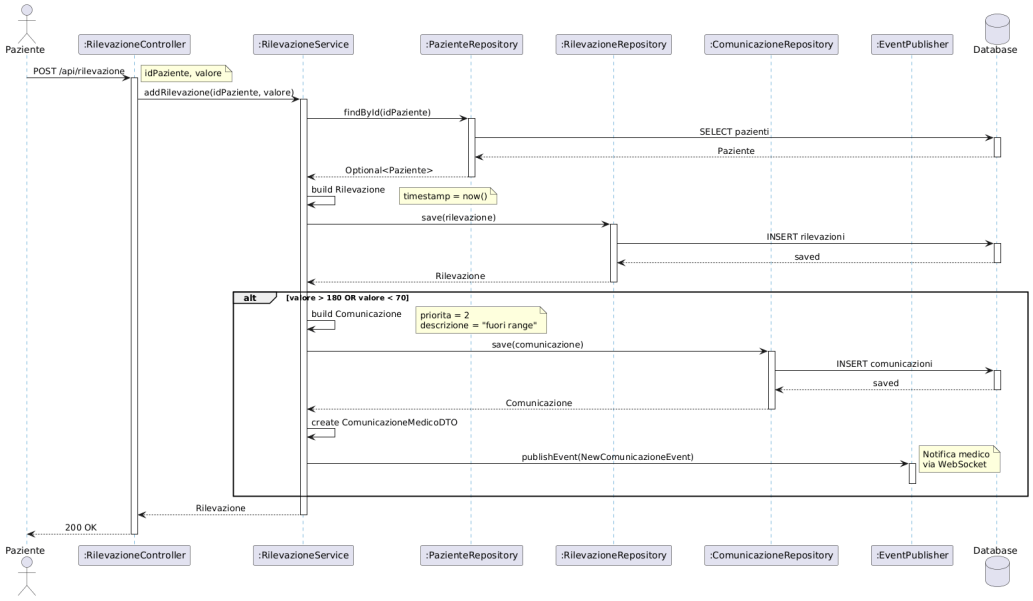


Figure 3.5: Diagramma di sequenza UML - Inserimento rilevazione

Logica di alert automatico:

- Range glicemico normale: 80-130 mg/dL
- Condizione alert: valore < 80 OR valore > 130 OR valore > 180 (se dopo i pasti)
- Se fuori range:
  - Crea Comunicazione con priorità 2 (media-alta)
  - Descrizione: "Paziente ha inserito rilevazione fuori range (valore: X)"
  - Salva comunicazione nel database
  - Pubblica NewComunicazioneEvent via ApplicationEventPublisher
  - Il medico riceve notifica real-time via WebSocket

Pattern utilizzati:

- Observer Pattern: pubblicazione evento per notifica asincrona
- Event-Driven Architecture: disaccoppiamento tra business logic e notifiche

3.2.5 Filtro JWT per autenticazione richieste

Ogni richiesta HTTP viene intercettata da JwtAuthenticationFilter che verifica la presenza e validità del token JWT, impostando il contesto di sicurezza Spring.

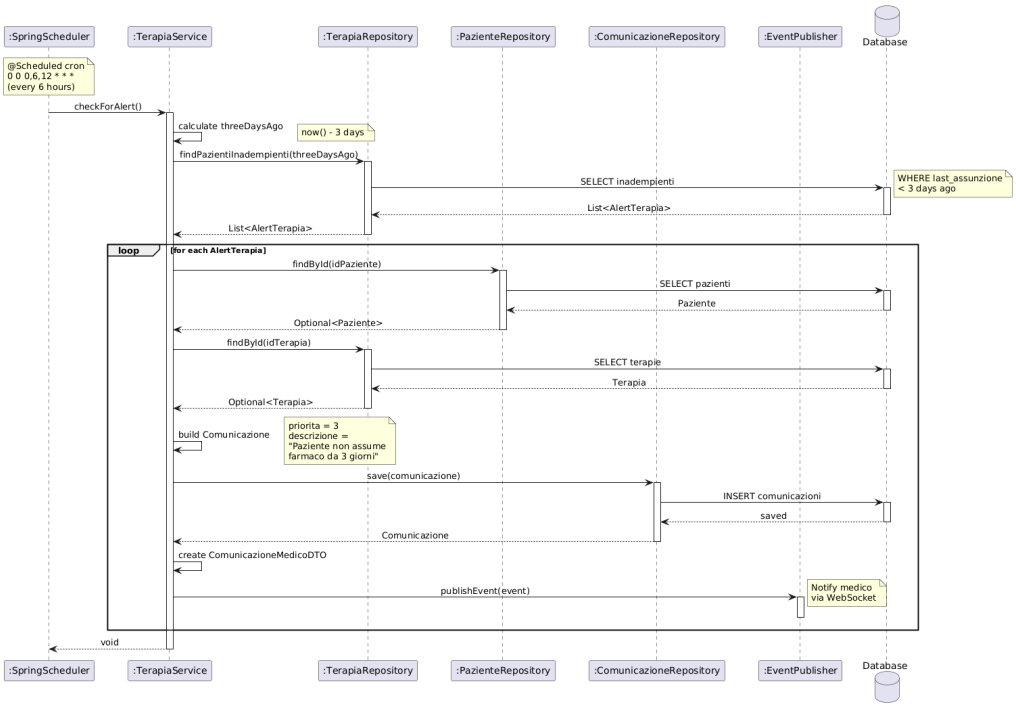


Figure 3.6: Diagramma di sequenza UML - Filtro JWT

Processo di verifica token:

1. Filter intercetta richiesta HTTP (OncePerRequestFilter di Spring)
2. Estrazione token dal cookie "token"
3. JwtService verifica:
   - Validità firma HMAC-SHA256
   - Data scadenza (expiry claim)
   - Assenza in blacklist (token non revocati)
4. Se valido:
   - Estrae email dal claim "sub"
   - Carica UserDetails dal database
   - Crea UsernamePasswordAuthenticationToken
   - Imposta SecurityContextHolder (contesto Spring Security)
5. Request procede nella FilterChain verso il Controller
6. Controller può accedere a Principal autenticato

Gestione casi edge:

- Token assente: request procede senza autenticazione
- Token invalido/scaduto: request procede senza autenticazione
- Endpoint pubblici (/api/auth/**): nessuna autenticazione richiesta

3.2.6 Logout

Il logout invalida il token JWT aggiungendolo a una blacklist persistente, impedendo riutilizzo anche se non ancora scaduto.

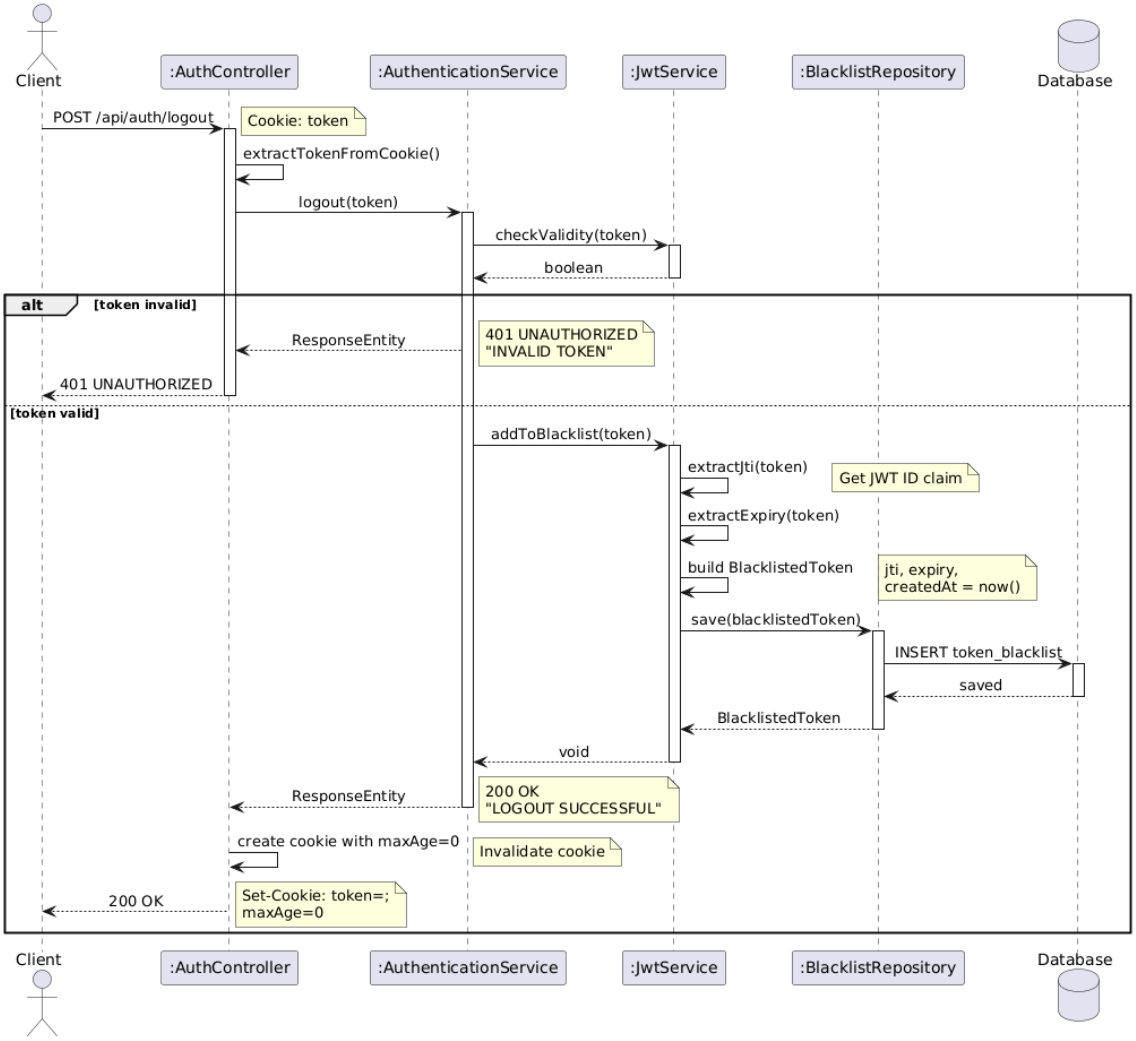


Figure 3.7: Diagramma di sequenza UML - Logout

Meccanismo blacklist:

1. Client invia POST /api/auth/logout con token in cookie
2. AuthController estrae token dal cookie

3. AuthenticationService verifica validità token
4. JwtService:
   - Estrae JWT ID univoco (claim "jti")
   - Estrae data scadenza (claim "exp")
   - Crea entità BlacklistedToken con (jti, expiry, createdAt=now())
   - Salva in tabella token_blacklist
5. AuthController crea cookie con maxAge=0 per invalidazione client-side
6. Response 200 OK "LOGOUT SUCCESSFUL"

Cleanup automatico:

- La blacklist contiene solo token non ancora scaduti
- Task schedulato (opzionale) rimuove token scaduti periodicamente
- Verifica blacklist avviene ad ogni richiesta autenticata

3.2.7 Controllo automatico terapie (Scheduled Task)

Un task schedulato con annotazione @Scheduled controlla periodicamente i pazienti che non hanno assunto farmaci da almeno 3 giorni, generando alert per i medici.

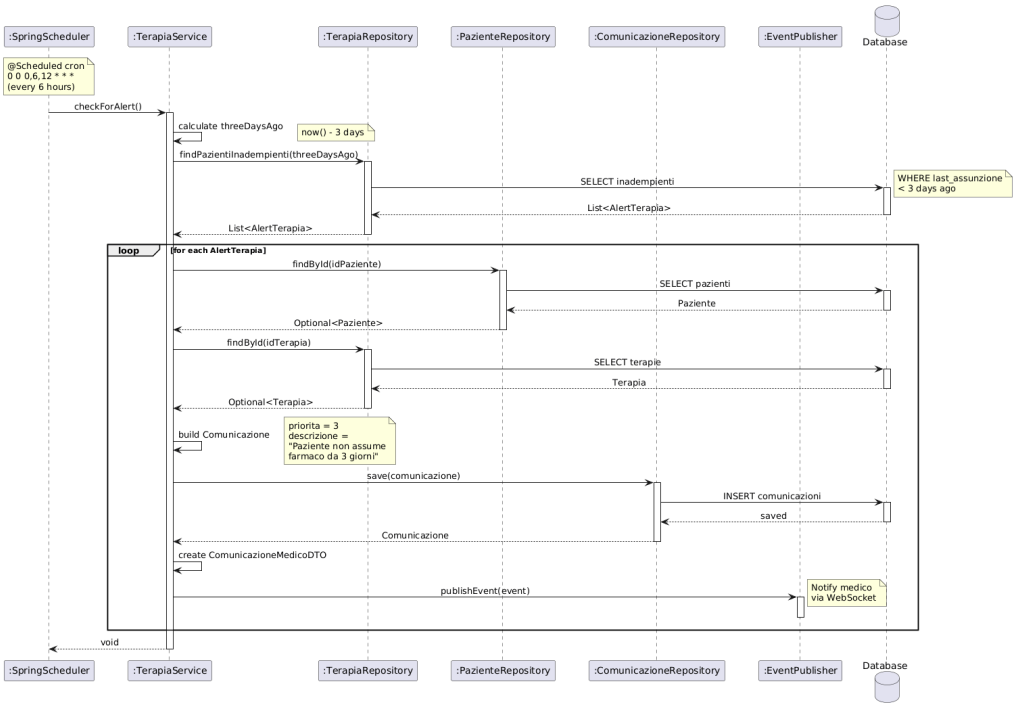


Figure 3.8: Diagramma di sequenza UML - Controllo automatico terapie

Configurazione scheduling:

- Annotazione: `@Scheduled(cron = "0 0 0,6,12 * * *")`
- Esecuzione: ogni 6 ore (mezzanotte, 6:00, mezzogiorno)
- Timezone: Europe/Berlin
- Metodo: `TerapiaService.checkForAlert()`

Logica di controllo:

1. Calcola threeDaysAgo = Instant.now() - Duration.ofDays(3)
2. Query custom TerapiaRepository.findPazientiInadempienti(threeDaysAgo)
3. Per ogni AlertTerapia trovato:
   - Recupera Paziente e Terapia dai repository
   - Crea Comunicazione con:
     - priorita = 3 (alta)
     - descrizione = "Paziente [nome] non assume [farmaco] da almeno 3 giorni"
     - timestamp = now()
   - Salva comunicazione nel database
   - Crea ComunicazioneMedicoDTO con dati paziente inclusi
   - Pubblica NewComunicazioneEvent(idMedico, DTO)
   - Medico riceve notifica real-time via WebSocket

Query custom (JPQL):

```
SELECT new AlertTerapia(t.id, p.id)
FROM Terapia t
JOIN t.idPaziente p
LEFT JOIN Assunzione a ON a.idTerapia = t
WHERE a.timestamp IS NULL
   OR a.timestamp < :threeDaysAgo
GROUP BY t.id, p.id
```

Chapter 4

Sviluppo del Software

4.1 Metodologia di sviluppo

Per lo sviluppo del sistema è stato adottato un modello di processo iterativo e incrementale di matrice Agile, caratterizzato da un'elevata flessibilità operativa in funzione delle specificità progettuali. Il workflow, iniziato con l'analisi dei requisiti e la progettazione architetturale preliminare, si è articolato nella pianificazione ed esecuzione di Sprint focalizzati sull'implementazione. Al termine di ogni iterazione, si è proceduto sistematicamente alla validazione tramite test, all'aggiornamento della documentazione tecnica e alla revisione dei requisiti, garantendo così un miglioramento continuo del software. Per la gestione del codice è stata utilizzata una repository Github per facilitare la collaborazione tra i componenti del team grazie alla possibilità di lavorare su branch separati.

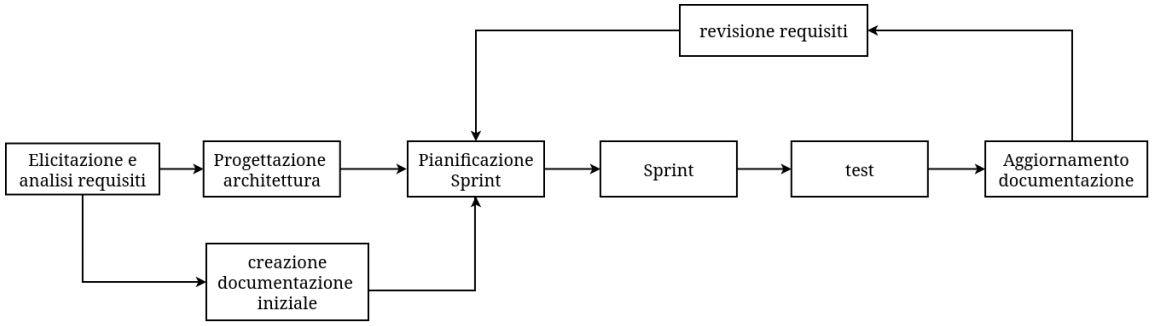


Figure 4.1: Processo di sviluppo

4.2 Architettura

Il software adotta un'architettura *Client-Server* con elementi del pattern *Model-View-Controller*. Il sistema si compone di:

- **Client (View)**: Interfaccia utente sviluppata in React 18 con TypeScript, responsabile della presentazione dati e dell'invio di richieste HTTP al server.
- **Server (Model + Controller)**: Implementato con Spring Boot (Java), suddiviso in:
  - **Controller**: Gestisce le richieste della web API tramite Rest Controller, elaborandole attraverso il modello dati e restituendo risposte JSON.
  - **Model**: Rappresenta i dati del sistema su H2 Database, gestendo la logica di business e le operazioni CRUD tramite Repository e Service.

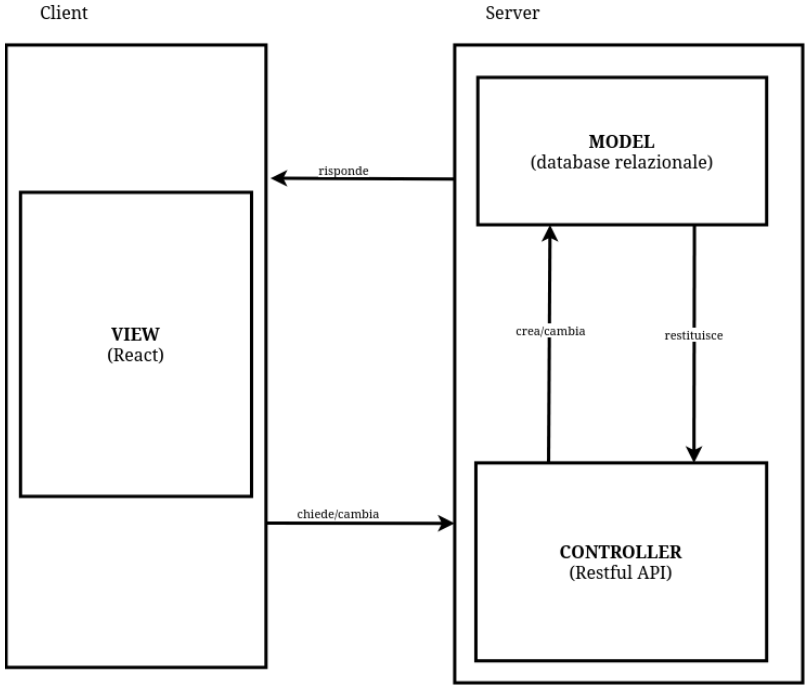


Figure 4.2: Architettura del sistema

4.2.1 Configurazione del Database

Il sistema utilizza **H2 Database**, un database relazionale embedded in Java, configurato in modalità persistenza. La gestione avviene tramite console Web o chiamate API.

Schema del Database

Il database contiene le seguenti tabelle principali, generate automaticamente da JPA:

- `utenti`: dati anagrafici e credenziali
- `pazienti`: estende utenti con dati specifici del paziente
- `rilevazioni`: valori glicemici registrati
- `terapie`: farmaci prescritti ai pazienti
- `assunzioni`: registrazioni delle assunzioni di farmaci
- `comunicazioni`: messaggi e alert tra paziente e medico
- `blacklisted_tokens`: token JWT invalidati
- `logs`: tracciamento operazioni di sistema

Le relazioni principali collegano pazienti a medici (ManyToOne), rilevazioni/terapie/assunzioni ai rispettivi pazienti, e comunicazioni tra pazienti e medici.

4.3 Pattern utilizzati

Durante lo sviluppo del sistema sono stati implementati diversi design pattern, ciascuno scelto per risolvere specifiche problematiche architetturali e migliorare la manutenibilità del codice.

4.3.1 Singleton

Il Singleton, implemetato implicitamente attraverso l'uso dell'annotazione @Service del framework è utilizzato per garantire che una classe servizio abbia una sola istanza e per fornire un unico punto di accesso globale a quell'istanza.

4.3.2 Repository Pattern

Implementato tramite Spring Data JPA, astrae l'accesso ai dati fornendo automaticamente metodi CRUD senza SQL manuale. Permette query personalizzate tramite metodi generati in utomatico dal framework Java Spring o JPQL, riducendo drasticamente il codice necessario.

4.3.3 Observer Pattern

Gestisce la comunicazione asincrona tramite il sistema di eventi di Spring. I Service pubblicano eventi significativi (es. rilevazioni anomale) che vengono intercettati da listener (es. WebSocket handler), garantendo disaccoppiamento tra logica di business e notifiche real-time.

4.3.4 Facade Pattern

I Service forniscono interfacce semplificate per operazioni complesse multi-sistema. Un esempio è il salvataggio di una rilevazione che orchestra automaticamente persistenza, verifica range, creazione alert, notifica WebSocket e logging.

4.3.5 Builder Pattern

Utilizzato per costruire oggetti complessi (entità JPA) in modo fluido tramite Lombok e `@Builder`, specificando solo i campi necessari ed evitando costruttori posizionali soggetti a errori.

4.4 Implementazione

4.4.1 Struttura del Backend (Spring Boot)

Il backend è organizzato seguendo una struttura modulare in package che favorisce la separazione delle responsabilità e la manutenibilità del codice:

- **entity**: contiene le entità JPA che mappano le tabelle del database. Ogni entità rappresenta un concetto del dominio applicativo (Utente, Paziente, Rilevazione, Terapia, Comunicazione, Assunzione)
- **repository**: interfacce che estendono `JpaRepository` per l'accesso ai dati. Spring Data JPA genera automaticamente l'implementazione dei metodi CRUD e delle query personalizzate
- **service**: contiene la logica di business e orchestra le operazioni tra repository e controller. I service gestiscono le transazioni e implementano le regole applicative
- **controller**: espone le REST API tramite `@RestController`. Ogni controller gestisce le richieste HTTP per una specifica area funzionale
- **dto**: Data Transfer Objects utilizzati per trasferire dati tra client e server, separando la rappresentazione interna (entity) da quella esposta
- **config**: configurazioni del sistema (sicurezza, JWT, WebSocket, CORS)
- **filter**: filtri per l'elaborazione delle richieste HTTP, in particolare il filtro JWT per l'autenticazione
- **forms**: classi per la validazione dei dati in ingresso dalle richieste
- **event**: gestione degli eventi applicativi per comunicazioni asincrone
- **exception**: gestione centralizzata delle eccezioni

4.4.2 Repository Layer

Il layer Repository utilizza Spring Data JPA per astrarre l'accesso al database. Ogni repository estende `JpaRepository<Entity, UUID>` che fornisce automaticamente:

- Metodi CRUD: `save()`, `findById()`, `findAll()`, `deleteById()`
- Query personalizzate tramite naming convention dei metodi
- Paginazione e ordinamento
- Gestione transazionale automatica

Repository implementati:

- `UtenteRepository`: gestione utenti con metodo `findByEmailAddress(String email)`
- `PazienteRepository`: operazioni specifiche sui pazienti
- `RilevazioneRepository`: query sulle rilevazioni con metodi come `findByIdPaziente_Id(UUID)` e `findAllByPazienteIdMedicoId(UUID)`
- `TerapiaRepository`: gestione terapie
- `AssunzioneRepository`: registrazione assunzioni
- `ComunicazioneRepository`: gestione comunicazioni
- `BlacklistedTokenRepository`: gestione token JWT invalidati

Spring Data JPA genera automaticamente l'implementazione basandosi sul nome del metodo, navigando le relazioni delle entità.

4.4.3 Service Layer

I Service orchestrano le operazioni tra Controller e Repository:

- **UtenteService**: Operazioni CRUD su utenti con validazione.
- **PazienteService**: Gestione pazienti inclusa assegnazione medico e aggiornamento dati clinici.
- **RilevazioneService**: Inserimento rilevazioni con controllo automatico range glicemici (alert se valore < 80 mg/dL o > 130 mg/dL o > 130 mg/dL dopo i pasti), creando comunicazioni automatiche con priorità alta e notifiche WebSocket.
- **TerapiaService**: Gestione completa terapie farmacologiche.
- **AssunzioneService**: Registrazione e storico assunzioni medicinali.
- **ComunicazioneService**: Messaggistica paziente-medico con gestione stati lettura.
- **AuthenticationService**: Gestisce registrazione (con form diversificati per ruolo, crittografia BCrypt, generazione JWT), autenticazione (verifica credenziali, generazione token, creazione cookie HTTP-only) e logout (invalidazione token via blacklist).
- **JwtService**: Gestione completa token JWT (generazione con claims email/ruolo/id, validazione firma e scadenza, verifica blacklist).

4.4.4 Controller Layer

I Controller espongono REST API mappando richieste HTTP ai Service tramite annotazioni `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`:

- **AuthController**: Endpoint per registrazione, login, logout e validazione token.

- **RilevazioneController**: Gestione rilevazioni con endpoint per inserimento, recupero (anche come DTO), eliminazione e visualizzazione filtrata per ruolo (admin vede tutto, medico solo suoi pazienti, paziente solo proprie).
- **PazienteController**: Operazioni su pazienti inclusa assegnazione medico.
- **TerapiaController, AssunzioneController, ComunicazioneController**: Gestione rispettive entità.
- **UtenteController**: Gestione utenti principalmente per admin.

4.4.5 Autenticazione e Sicurezza

Il sistema implementa autenticazione JWT tramite Spring Security con componenti:

- **JwtAuthenticationFilter**: Filtro che intercetta ogni richiesta, estrae token JWT dal cookie, valida tramite JwtService, carica dettagli utente e imposta autenticazione nel SecurityContext.
- **Token JWT**: Contiene header (algoritmo HS256), payload (subject email, claims id/ruolo, issued at, expiration) e firma HMAC. La generazione avviene tramite metodo builder di Jwts, la validazione verifica firma, scadenza e blacklist.
- **Flusso autenticazione**: Login valida credenziali con BCrypt, genera token JWT, crea cookie HTTP-only; richieste successive includono cookie che viene validato dal filtro; logout aggiunge token a blacklist.
- **Gestione ruoli**: ROLE_ADMIN (accesso completo), ROLE_MEDICO (gestione pazienti assegnati, terapie, comunicazioni), ROLE_PAZIENTE (solo dati personali). Controlli implementati nei Service con logica condizionale basata su ruolo.
- **Sicurezza aggiuntiva**: Password con BCrypt mai in chiaro, campo `@JsonIgnore`; token blacklist per logout sicuro; cookie HTTP-only contro XSS.

4.4.6 Comunicazione Real-time con WebSocket

Il sistema implementa WebSocket per notifiche real-time tramite due handler:

- **ComunicazioneWebSocketHandler**: Gestisce connessioni persistenti medico-server usando mappa concorrente delle sessioni. Ascolta eventi `NewComunicazioneEvent` e invia notifiche JSON immediate al medico interessato.
- **LogsWebSocketHandler**: Fornisce streaming real-time dei log di sistema agli admin connessi per monitoraggio operazioni e debugging.
- **Configurazione**: Classe config con `@EnableWebSocket` registra handler su endpoint `comunicazioni:?id=idMedico` e `/ws/logs`.
- **Flusso notifiche**: Rilevazione anomala → RilevazioneService crea Comunicazione → Pubblica evento → Handler cattura e invia via WebSocket → Frontend mostra alert real-time. Sistema basato su eventi garantisce disaccoppiamento tra logica business e notifiche.

4.4.7 Data Transfer Objects (DTO)

I DTO separano la rappresentazione interna (Entity) da quella esterna (API).

DTO principali:

- **RilevazioneUtenteDTO**: rilevazione con dati utente inclusi
- **ComunicazioneMedicoDTO**: comunicazione con info paziente
- **PazienteForm**: validazione dati registrazione paziente
- **MedicoForm**: validazione registrazione medico
- **SignInForm**: validazione credenziali login

Vantaggi dei DTO:

- Evitano esposizione di dati sensibili (es. password)
- Prevengono serializzazione ricorsiva di relazioni
- Permettono validazione specifica per operazione
- Riducono payload di rete (solo campi necessari)
- Disaccoppiano API da struttura database

4.4.8 Gestione delle eccezioni

Il sistema implementa gestione centralizzata degli errori.

Meccanismi:

- `ResponseStatusException`: eccezioni con HTTP status
- Try-catch nei service con logging
- Propagazione eccezioni ai controller
- Risposta JSON strutturata per errori

4.4.9 Logging

Sistema di logging implementato con SLF4J e Logback.

Livelli di log utilizzati:

- `INFO`: operazioni normali (login, registrazione, operazioni CRUD)
- `WARN`: situazioni anomale ma gestibili
- `ERROR`: errori che impediscono operazioni

Esempi:

```
logger.info("Tentativo di login per email: " + email);
logger.warn("Tentativo di login con email non esistente");
logger.error("Errore durante salvataggio: " + e.getMessage());
```

4.4.10 Struttura del Frontend (React + TypeScript)

Il frontend è sviluppato con React 18 e TypeScript, organizzato per favorire riusabilità e manutenibilità.

Struttura directory principale:

- `src/components/`: componenti React riutilizzabili
- `src/pages/`: pagine principali dell'applicazione
- `src/services/`: moduli per chiamate API
- `src/context/`: Context API per stato globale
- `src/hooks/`: custom hooks
- `src/types/`: definizioni TypeScript
- `src/utils/`: funzioni di utilità

Gestione dello stato globale

Utilizzo di React Context API per stato condiviso:

AuthContext:

- Gestisce stato autenticazione
- Memorizza token JWT
- Fornisce funzioni login/logout
- Protegge route private

UserContext:

- Memorizza dati utente loggato
- Fornisce info ruolo per rendering condizionale
- Gestisce aggiornamento profilo

Comunicazione con il Backend

Tutte le chiamate API sono gestite tramite Axios con configurazione centralizzata:

Interceptor richieste:

- Aggiunge automaticamente credenziali (cookie con token)
- Imposta headers comuni
- Logging richieste in sviluppo

Interceptor risposte:

- Gestione errori centralizzata
- Redirect a login se 401 Unauthorized
- Toast notification per errori

Type Safety con TypeScript

Definizione di interfacce per type safety completo:

```
interface Utente {
    id: string;
    email: string;
    nome: string;
    cognome: string;
    dataNascita: string;
    ruolo: 'ROLE_PAZIENTE' | 'ROLE_MEDICO' | 'ROLE_ADMIN';
}

interface Rilevazione {
    id: string;
    idPaziente: string;
    valore: number;
    timestamp: string;
}
```

Routing e navigazione

React Router per navigazione SPA:

- Route pubbliche: login, registrazione
- Route protette: dashboard, profilo, rilevazioni
- Guard basati su AuthContext
- Redirect automatico se non autenticati

Chapter 5

Test

In questa fase del progetto ci siamo assicurati che tutte le funzionalità richieste fossero implementate e che i vari diagrammi fossero coerenti.
Successivamente abbiamo poi creato dei test automatizzati utilizzando JUnit e Mockito con l'obbiettivo di verificare automaticamente il corretto funzionamento di alcuni parti del codice.
Dopo l'esecuzione degli Unit test abbiamo svolto un test manuale dell'applicazione, e infine, abbiamo fatto provare la webapp ad alcuni utenti esterni per ottenere un feedback da parte di un possibile utente della webapp.

5.1 Test automatici

I nostri test automatici controllano il funzionamento dei service, ovvero delle funzionalità dell'applicazione e delle possibilità offerte agli utenti.

In questi test come detto in precedenza abbiamo usato Mock per simulare repository e dipendenze necessarie per testare automaticamente i seguenti service:

- GlicemiawebApplicationTests.java : il test principale che verifica il corretto avvio del contesto di spring boot
- AssunzioneServiceTest.java : testa le funzionalità legate alle assunzioni di medicinali, cioè la possibilità del sistema di recuperare i dati necessari e la possibilità del paziente di registrarli
- AuthenticationServiceTest.java : viene testata l'autenticazione, sia per login che logout grazie all'utilizzo di un token JWT.
- ComunicazioneServiceTest.java : testa la comunicazione paziente-medico, si verifica la funzionalità di invio e ricezione dei messaggi e pure la possibilità del medico di segnare come letti i messaggi ricevuti
- PazienteServiceTest.java : testa delle funzionalità legate alla gestione del paziente, come la possibilità di recuperare i pazienti anche in base al medico, viene testata anche la possibilità di aggiornare i dati del paziente, l'associazione del medico e la gestione della password.

- RilevazioneServiceTest.java : testa l'inserimento e l'eliminazione di una rilevazione da parte dell'utente, per la visualizzazione dei dati viene pure testata la possibilità di recuperarli ma con l'applicazione di filtri basati sul ruolo
- TerapiaService.java : testa la gestione delle terapie, quindi la creazione, modifica e l'eliminazione da parte del medico.
- UtenteServiceTest.java : testa la gestione di tutti gli utenti, quindi la creazione, la modifica e l'eliminazione di medici e pazienti.

5.2 Test manuali

Abbiamo poi eseguito dei test manuali per verificare il corretto funzionamento della applicazione, utilizzando software come Postman o Insomnia per testare le API e tramite interfaccia grafica per della webapp seguendo l'utilizzo che ne farebbe un utente non sviluppatore. di seguito le principali verifiche:

- Verifica del corretto funzionamento dell'autenticazione
- Verifica del corretto funzionamento di tutte le funzionalità utente per utente
- Verifica di mancato salvataggio in caso di dati non validi o vuoti

5.2.1 Test di un utente reale

Infine abbiamo sottoposto degli utenti esterni alla nostra applicazione per avere un un feedback da qualcuno di esterno che potrebbe essere un possibile utilizzatore dell'applicazione. In questa fase siamo stati solo osservatori e non abbiamo dato alcun tipo di informazione su come utilizzare l'applicazione eccetto per le credenziali di accesso e lo scopo della applicazione. Grazie a questo test abbiamo migliorato l'applicazione in termini di user friendly e a livello grafico.